

Arquitectura de bases de datos espaciales y estándares OGC

i Entorno de ejecución interactivo: Las consultas y sentencias DDL/DML de este capítulo asumen que la sesión de terminal dispone de las variables de conexión estándar de `libpq` debidamente configuradas. Si ejecuta los bloques de forma manual en la consola, verifique la asignación de parámetros descrita en la `?@sec-gestion-variables-entorno`.

Objetivos del capítulo

- Analizar los **fundamentos** conceptuales de las bases de datos y evaluar sus **ventajas** determinantes frente a los sistemas de archivos tradicionales.
- Comprender la **arquitectura** funcional, los **niveles de abstracción** y los subsistemas operativos de un sistema gestor de bases de datos (SGBD).
- Determinar el **modelo de datos y la arquitectura** de integración óptimos para soportar aplicaciones de sistemas de información geográfica (SIG).
- Examinar la implementación de los **estándares** internacionales del Open Geospatial Consortium (OGC) e ISO en motores relacionales extendidos.

De sistemas de archivos a bases de datos relacionales

El manejo de información en organizaciones y proyectos de geomática ha transitado históricamente desde esquemas basados en sistemas de archivos independientes hacia arquitecturas centralizadas administradas por **sistemas gestores de bases de datos (SGBD)** (*Database Management System*, DBMS).

Un sistema de archivos tradicional almacena los datos en estructuras cerradas dependientes de aplicaciones específicas (archivos planos, hojas de cálculo o formatos propietarios). Este enfoque presenta deficiencias estructurales críticas:

- **Redundancia e inconsistencia:** La duplicación descontrolada de archivos genera discrepancias en el estado de la información cuando se ejecutan actualizaciones parciales o no coordinadas.
- **Aislamiento y fragmentación de datos:** La dispersión de formatos y codificaciones heterogéneas impide la interoperabilidad y dificulta la formulación de consultas analíticas integradas.
- **Dependencia programa-datos:** Las aplicaciones se acoplan rígidamente a la estructura física de almacenamiento, lo que exige reescribir el software de consumo ante cualquier cambio en el formato de los archivos.

- **Vulnerabilidad transaccional y concurrencia deficiente:** Carencia de mecanismos ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), lo que provoca condiciones de carrera, sobreescritura destructiva y corrupción de datos ante fallos del sistema o accesos simultáneos.
- **Costos elevados de mantenimiento y gobierno:** La ausencia de esquemas y catálogos normalizados fragmenta las políticas de seguridad, integridad y respaldo.

Frente a estas restricciones, una **base de datos** se define como una colección **integrada, estructurada y no redundante** de datos almacenados en soporte **persistente**, accesible **concurrentemente** por múltiples usuarios y aplicaciones mediante una estricta separación entre la representación **lógica** y la implementación **física**. Por su parte, el **sistema gestor de bases de datos (SGBD)** (*Database Management System*, DBMS) constituye la infraestructura de **software** responsable de proveer las interfaces y mecanismos para la **definición, manipulación, optimización, persistencia y administración** integral de dicha información.

Cuando el motor relacional se extiende con soporte nativo para tipos **geométricos** y **geográficos**, predicados **topológicos** y métodos de **indexación multidimensional**, se conforma un **SGBD espacial** (*Spatial Database Management System*, SDBMS). En este texto se adoptan como sinónimos técnicos los términos **base de datos geográfica**, **base de datos espacial** o **GeoDB** para designar cualquier repositorio estructurado con soporte nativo de tipos espaciales. Por convención académica, el término comercial **geodatabase** se reserva exclusivamente para las estructuras de almacenamiento propietarias del ecosistema ESRI. Las definiciones complementarias se consolidan en el glosario técnico al cierre del capítulo.

Para comprender con rigor esta arquitectura, es necesario distinguir entre el plano **técnico** de habilitación y el principio **arquitectónico** fundamental. Extender el motor relacional con tipos geométricos, índices multidimensionales y predicados topológicos constituye el medio técnico instrumental. No obstante, la verdadera ruptura conceptual de un SGBD espacial radica en la **integración y centralización de esquemas**: la unificación de la geometría como un objeto de primera clase dentro del núcleo transaccional.

Como se sintetiza en la Figure 1, el motor gestor consolida en un repositorio común **atributos descriptivos**, **geometrías vectoriales tipadas** y **metadatos de referencia espacial**, gobernados por un **catálogo centralizado** y un **subsistema de integridad transaccional** accesible por múltiples clientes en red.

Componentes y funciones de un SGBD

La arquitectura de un SGBD desacopla el almacenamiento físico de la interfaz lógica del usuario para garantizar concurrencia, rendimiento e integridad de la información.

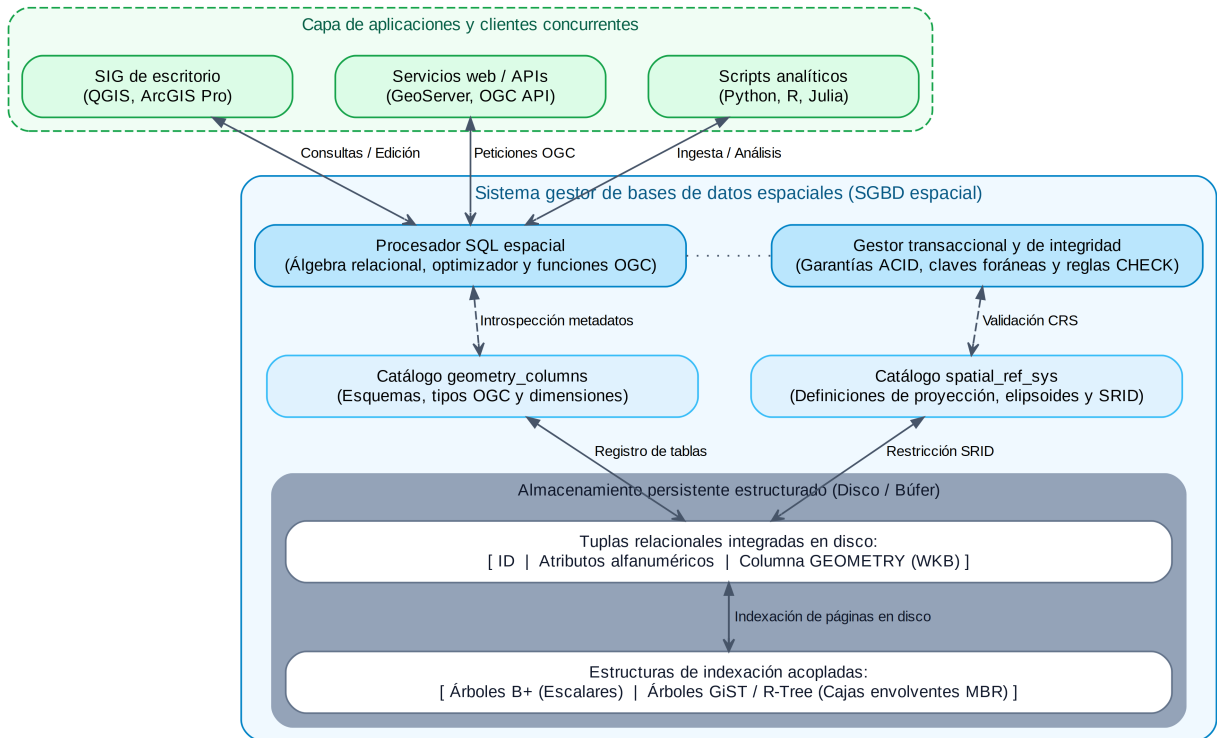


Figure 1: Centralización e integración de datos espaciales y alfanuméricos en un SGBD espacial.

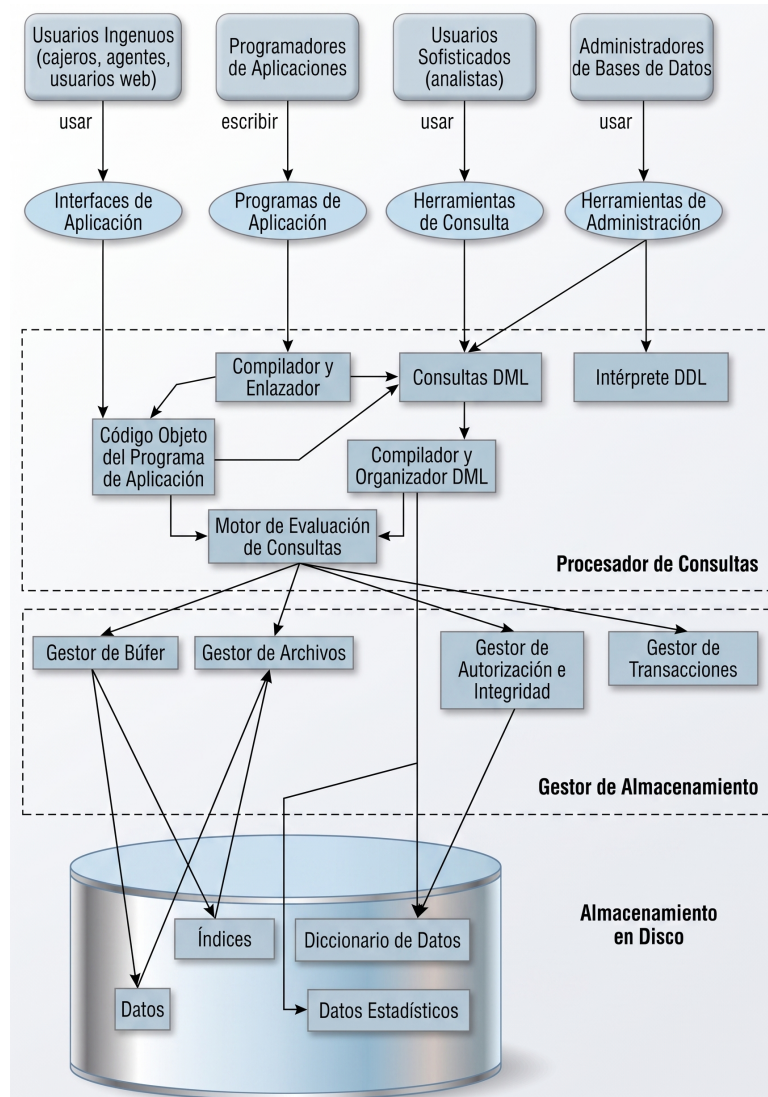


Figure 2: Componentes de un sistema de bases de datos, según Silberschatz, Korth y Sudarshan [-@silberschatz2002database].

En la Figure 2 se presenta la arquitectura clásica funcional de un SGBD estructurada en tres capas operativas fundamentales: el **procesador de consultas**, el **gestor de almacenamiento** y el **almacenamiento en disco**.

Procesador de consultas:

El **procesador de consultas** constituye el subsistema responsable de transformar instrucciones declarativas de alto nivel (ej. sentencias SQL donde se especifica qué información se requiere y no el algoritmo para buscarla) en planes algebraicos de ejecución de bajo costo, coordinando su despacho hacia el gestor de almacenamiento. Las sentencias SQL ingresan a este procesador a través de dos vías fundamentales: como **SQL estático embebido**, donde las consultas forman parte integral del código fuente (ej. programas en C, C++ o COBOL) de un programa de aplicación (ej. un software GIS de escritorio o un sistema catastral desarrollado a medida) y son analizadas durante la fase de compilación y enlazado para generar código objeto ejecutable (ej. código binario de máquina vinculado a librerías cliente dinámicas `.dll` o `.so`); o como **SQL dinámico**, donde las sentencias se generan, concatenan o parametrizan al vuelo durante el tiempo de ejecución desde herramientas interactivas o interfaces de usuario. En ambos casos, el procesador debe interpretar la sintaxis, validar los esquemas contra el catálogo, optimizar el árbol de operadores relacionales y ejecutar las instrucciones físicas resultantes.

- **Intérprete DDL:** Procesa las sentencias de definición de datos y registra las definiciones correspondientes en el diccionario de datos (vínculo que la arquitectura conceptual formaliza directamente, aunque se omita la línea en el diagrama de Silberschatz).
- **Compilador y organizador DML:** Traduce las sentencias DML formuladas en lenguajes de consulta a un plan de evaluación de bajo nivel compuesto por operadores relacionales.
- **Motor de evaluación de consultas:** Ejecuta las instrucciones de bajo nivel generadas por el compilador DML interactuando directamente con el gestor de almacenamiento.
- **Compilador y enlazador de programas de aplicación:** Integra las llamadas a funciones DML embebidas en el código fuente con el motor de evaluación para generar código objeto ejecutable.

Gestor de almacenamiento:

El **gestor de almacenamiento** es el responsable de proveer la interfaz operativa entre los datos de bajo nivel almacenados en el medio secundario persistente y las instrucciones lógicas despachadas por el motor de evaluación de consultas. Su función cardinal radica en abstraer las complejidades del hardware y del sistema de archivos del sistema operativo, traduciendo las peticiones de tuplas, geometrías e índices en transferencias eficientes de entrada/salida (I/O) de bloques o páginas hacia la memoria RAM. Al mismo tiempo, este módulo coordina la sincronización de accesos concurrentes, salvaguarda la durabilidad y consistencia transaccional mediante bitácoras en disco, y valida que las operaciones de inserción, actualización o consulta cumplan con las directivas de seguridad y las restricciones de dominio e integridad referencial definidas en el catálogo.

- **Gestor de autorización e integridad:** Valida las restricciones de integridad declarativas y comprueba los privilegios de acceso de los usuarios a partir de los datos estadísticos y el diccionario.
- **Gestor de transacciones:** Garantiza que la base de datos permanezca en un estado consistente a pesar de fallos del sistema, coordinando la ejecución concurrente sin interferencias.
- **Gestor de archivos:** Asigna y gestiona el espacio en el almacenamiento secundario y las estructuras de datos usadas para representar los registros.
- **Gestor de búfer:** Controla la transferencia de páginas entre la memoria volátil intermedia (*buffer pool*) y el almacenamiento en disco persistente.

Estructuras de almacenamiento en disco:

Las **estructuras de almacenamiento** en disco constituyen el estrato de persistencia física no volátil del SGBD, encargado de alojar de forma permanente tanto los registros de información como las estructuras de control requeridas para el funcionamiento del motor. Este nivel organiza el almacenamiento secundario en bloques o páginas de tamaño fijo, desacoplando la representación lógica de las tablas de su almacenamiento físico subyacente. En este sustrato conviven no solo los datos crudos del usuario y las geometrías serializadas, sino también los métodos de acceso acelerado (árboles B+, índices espaciales GiST o R-Tree), los catálogos del sistema que formalizan los esquemas e invariantes del modelo, y los metadatos estadísticos empleados por el optimizador de consultas para estimar costos de entrada y salida (I/O).

- **Datos:** Bloques físicos que alojan los registros de la base de datos.
- **Índices:** Estructuras de acceso rápido para acelerar la localización de tuplas específicas.
- **Diccionario de datos:** Almacén centralizado de metadatos sobre la estructura lógica y esquemas del sistema.
- **Datos estadísticos:** Información cuantitativa del catálogo sobre la distribución de las relaciones, empleada por el optimizador de consultas.

En la Table 1 se detalla la correspondencia entre los módulos de la arquitectura de Silberschatz, su nivel operativo y su responsabilidad fundamental dentro del motor de base de datos.

Table 1: Componentes principales del SGBD según Silberschatz y sus responsabilidades funcionales.

Componente	Nivel funcional	Responsabilidad principal
Intérprete DDL	Procesamiento lógico	Procesamiento de sentencias de definición y actualización de metadatos.
Compilador y organizador DML	Procesamiento lógico	Traducción de consultas de alto nivel a planes de ejecución evaluables.

Componente	Nivel funcional	Responsabilidad principal
Motor de evaluación de consultas	Ejecución lógica	Orquestación y despacho de instrucciones de bajo nivel al motor de almacenamiento.
Gestor de autorización e integridad	Almacenamiento / Control	Validación de restricciones declarativas y verificación de permisos.
Gestor de transacciones	Transaccional	Preservación de propiedades ACID y control de operaciones simultáneas.
Gestor de búfer	Almacenamiento físico	Paginación y caché de bloques de memoria entre disco y RAM.
Gestor de archivos	Almacenamiento físico	Asignación de espacio físico y direccionamiento de bloques de registros e índices.

Como marco comparativo, Elmasri y Navathe [-@ramez2007fundamentos] plantean una descomposición modular con mayor detalle en los subsistemas de ejecución de transacciones y el procesador de base de datos en tiempo de ejecución (*runtime database processor*), tal como se ilustra en la Figure 3.

Importancia y características de los SGBD

En la infraestructura tecnológica contemporánea, los SGBD constituyen el núcleo operativo para garantizar la persistencia, disponibilidad e integridad de los activos de información. Su arquitectura supera la fragilidad intrínseca de los sistemas de archivos mediante la implementación de niveles formales de abstracción de datos (esquemas físico, lógico y de vistas, formalizados en la Section), lo que garantiza el desacoplamiento estricto entre las aplicaciones cliente y las modificaciones en las estructuras de almacenamiento subyacentes.

Las propiedades funcionales y arquitectónicas que distinguen a un SGBD frente a soluciones ad-hoc (a la medida) comprenden:

- **Independencia lógica y física:** Capacidad de modificar la organización física de los archivos o el diseño conceptual de las tablas sin requerir la reescritura de las aplicaciones que consumen la base de datos.
- **Soporte de tipado abstracto y dominios complejos:** Definición rigurosa de tipos primitivos (escalares, numéricos, temporales) y estructurados (JSON, tipos geométricos/geográficos OGC, grandes objetos binarios o BLOB).

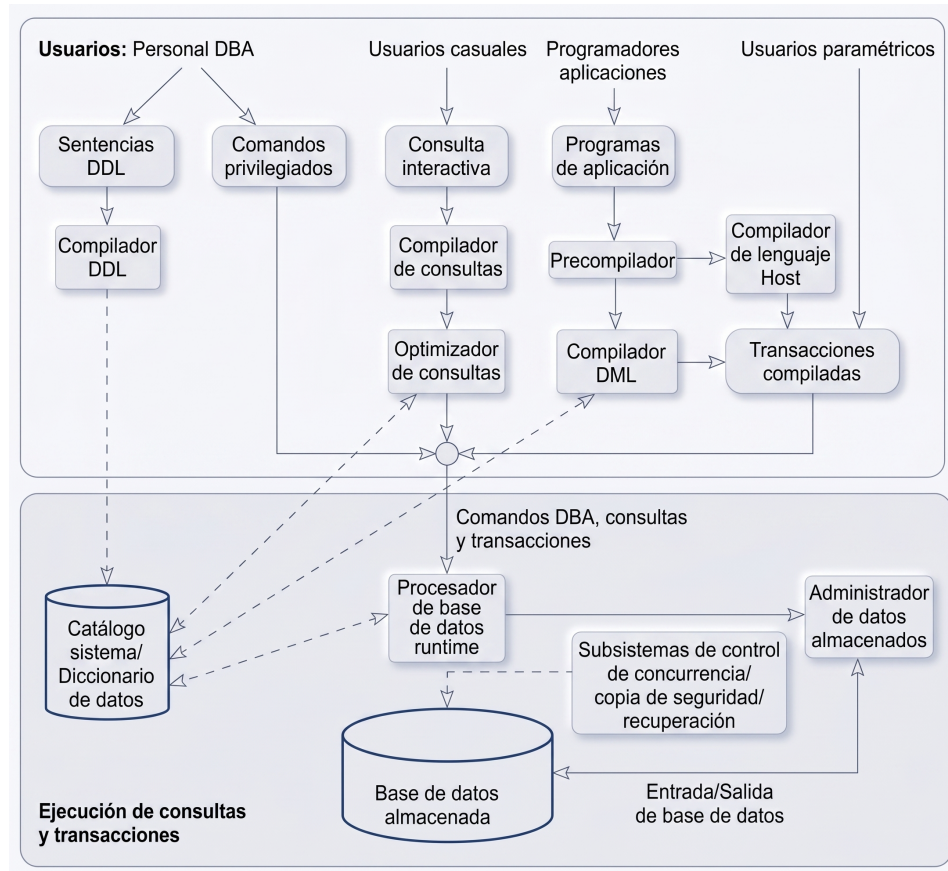


Figure 3: Arquitectura de componentes y ejecución de transacciones, según Elmasri y Navathe [-@ramez2007fundamentos].

- **Procesamiento declarativo y optimización de consultas:** Interpretación de sentencias en SQL estándar mediante un optimizador basado en costos (CBO), el cual formula planes de ejecución eficientes a partir de la distribución estadística de los datos.
- **Mecanismos de concurrencia y garantías ACID:** Aislamiento de transacciones concurrentes para evitar anomalías como lecturas sucias, actualizaciones perdidas y lecturas no repetibles.
- **Estructuras de acceso optimizado:** Implementación de índices univariados y multidimensionales (árboles B+, GiST, SP-GiST, R-Trees) que reducen exponencialmente los costos de entrada/salida (I/O) en operaciones de filtrado y agregación.
- **Tolerancia a fallos y recuperación:** Persistencia transaccional apoyada en registros de auditoría y bitácoras de operaciones (*Write-Ahead Logging*), asegurando la reconstrucción del estado consistente del sistema ante interrupciones no programadas de energía o caídas del kernel.
- **Seguridad y auditoría granular:** Control de accesos basado en roles (RBAC), mecanismos de autenticación criptográfica, cifrado en reposo y en tránsito, y políticas de seguridad a nivel de fila (*Row-Level Security*).

Perfiles de usuario y responsabilidades del administrador

La arquitectura canónica formaliza cuatro categorías de usuarios en función de su nivel de interacción técnica y el grado de abstracción con el que operan:

- **Usuarios ingenuos o paramétricos:** Interactúan con el sistema de forma indirecta mediante interfaces de usuario precompiladas y programas de aplicación (formularios web, aplicaciones móviles o terminales transaccionales), ejecutando operaciones predefinidas sin conocer la sintaxis SQL ni el modelo de datos subyacente.
- **Programadores de aplicaciones:** Desarrollan artefactos de software en lenguajes anfitriónes (Python, Java, C++, Julia), embebiendo sentencias DML o invocando interfaces de programación (ODBC, JDBC, APIs nativas) para conectar las aplicaciones con el motor de base de datos.
- **Usuarios sofisticados o analistas:** Interactúan directamente con el procesador de consultas mediante herramientas interactivas, consolas SQL o plataformas de ciencia de datos, formulando consultas analíticas complejas y extrayendo conjuntos de datos sin intermediación de código de aplicación estático.
- **Usuarios especializados:** Diseñan e implementan sistemas de bases de datos no tradicionales, tales como motores de bases de conocimiento, procesadores analíticos multidimensionales o sistemas de información espacial avanzados.

Dentro de este ecosistema, el **administrador de bases de datos (DBA)** representa la figura de control centralizado encargada del ciclo de vida operativo, el rendimiento y la seguridad del sistema gestor. Sus responsabilidades comprenden:

- **Definición del esquema conceptual y físico:** Redacción de especificaciones en lenguaje DDL para la creación del modelo relacional, las cuales el compilador DDL interpreta y almacena en el diccionario de datos.
- **Selección de métodos de acceso y estructuras de almacenamiento:** Definición de la topología de almacenamiento físico, asignación de espacios de tablas (*tablespaces*), tamaño de páginas de disco y diseño de índices primarios y secundarios.
- **Modificación y afinamiento físico (*tuning*):** Ajuste dinámico de parámetros del gestor de búferes, memoria intermedia y reestructuración periódica de índices para mitigar cuellos de botella según el comportamiento de la carga de trabajo.
- **Gestión de autorizaciones y directivas de seguridad:** Creación de roles de usuario, asignación granular de privilegios sobre esquemas o tablas, y administración de credenciales bajo el principio de mínimo privilegio.
- **Supervisión de restricciones de integridad:** Especificación de reglas declarativas (claves primarias, integridad referencial, restricciones de dominio) para que el gestor de integridad impida estados inconsistentes.
- **Mantenimiento preventivo, respaldo y contingencia:** Definición de políticas de copias de seguridad (completas, incrementales y continuas), monitorización del almacenamiento secundario y ejecución periódica de simulacros de recuperación ante desastres.

Niveles de abstracción e independencia de datos

Un sistema gestor de bases de datos abstrae la complejidad del almacenamiento secundario para presentar una representación simplificada de la información. La arquitectura estándar de tres niveles (ANSI/SPARC) formaliza este principio mediante capas operativas independientes, como se ilustra en la [Figure 4](#).

Las tres capas estructuran responsabilidades específicas:

1. **Nivel físico o interno:** Describe la organización técnica en que los registros se estructuran en el disco: distribución de páginas de datos, métodos de compresión, rutas de acceso físicas y estructuras de indexación secundarias (árboles B+ o árboles R/GiST en contextos espaciales).
2. **Nivel conceptual o lógico:** Modela la estructura integral del dominio de datos con independencia de los detalles del hardware de almacenamiento. En este nivel se definen las entidades, atributos, dominios (incluyendo tipos geométricos OGC), restricciones de integridad (CHECK, claves primarias, llaves foráneas) y relaciones lógicas entre conjuntos de datos.
3. **Nivel de visión o externo:** Define representaciones parciales y personalizadas del esquema lógico orientadas a distintos perfiles de usuario, roles operativos o módulos de software mediante vistas relacionales (VIEW) o políticas de seguridad a nivel de filas y columnas.

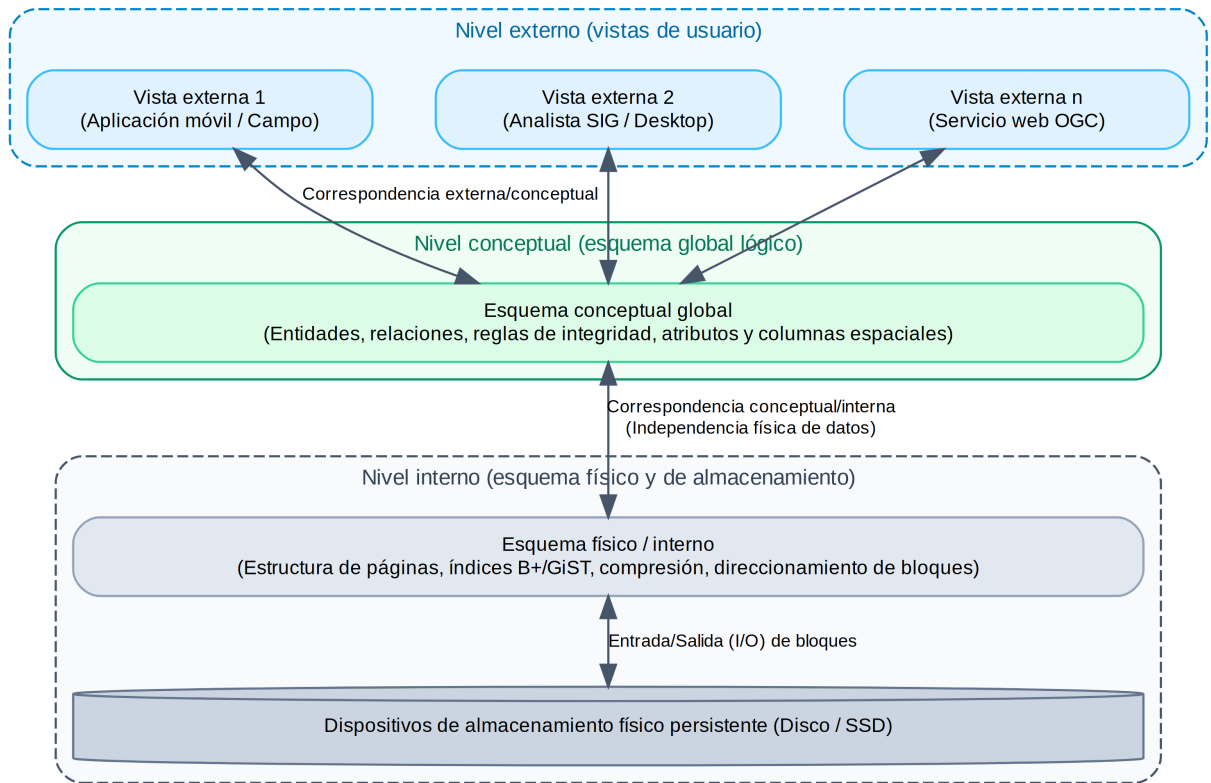


Figure 4: Arquitectura ANSI/SPARC de tres niveles y correspondencias de independencia de datos.

En la Table 2 se sintetizan las responsabilidades de cada estrato arquitectónico, los perfiles de usuario involucrados y su manifestación técnica en motores geospaciales.

Table 2: Niveles de abstracción arquitectónica y su instanciación técnica.

Nivel de abstracción	Audiencia principal	Mecanismos de implementación	Instanciación espacial (SGBD espacial)
Visión (externo)	Usuarios finales y analistas	Vistas SQL (VIEW , MATERIALIZED VIEW), permisos de rol.	Capas filtradas por cobertura territorial o temática.
Conceptual (lógico)	Administrador de datos y diseñador	Esquemas DDL, restricciones relacionales, tipos de datos.	Tipos de datos espaciales (GEOMETRY , GEOGRAPHY , SRID).
Físico (interno)	DBA y motor de almacenamiento	Asignación de páginas en disco, bloques físicos, WAL.	Codificación binaria (WKB) e índices espaciales (GiST, SP-GiST).

La separación desacoplada entre estos estratos garantiza dos propiedades operativas fundamentales:

- **Independencia física de datos:** Capacidad de reestructurar la disposición de almacenamiento, cambiar de medio físico, modificar el particionamiento de tablas o crear nuevos índices en disco sin requerir ajustes en el esquema conceptual ni reescribir las sentencias de consulta de las aplicaciones cliente.
- **Independencia lógica de datos:** Capacidad de expandir o modificar el esquema conceptual (añadir nuevas columnas, entidades o relocalizar tablas) sin alterar las aplicaciones que consumen vistas externas preexistentes, siempre que estas no dependan de los elementos modificados.

Evolución de modelos de datos y la dimensión espacial

A lo largo del desarrollo de los sistemas de información, los modelos de bases de datos se han estructurado en torno a paradigmas que responden tanto a las limitaciones del hardware de cada época como a la complejidad semántica de los datos:

- **Modelos basados en registros y punteros físicos (Tipo 1):** Dominantes en las décadas de 1950 y 1960 (modelos jerárquicos y de red), donde las asociaciones entre entidades se implementaban mediante enlaces físicos de memoria o apuntadores en disco.

- **Modelo relacional (Tipo 2):** Introducido por E. F. Codd (@codd1970relational), fundamentado en la teoría de conjuntos y la lógica de predicados de primer orden. La información se modela en relaciones (tablas compuestas por tuplas y atributos atómicos), y las vinculaciones se realizan exclusivamente a través de valores escalares compartidos (claves primarias y foráneas).
- **Modelos objeto-relacionales y orientados a objetos (Tipo 3):** Formulados a partir de 1990 para superar la incapacidad del modelo relacional plano al manejar datos no atómicos. Introducen encapsulamiento, herencia, métodos y soporte nativo para tipos abstractos de datos (ADT), habilitando la persistencia e indexación de geometrías espaciales y tipos de datos multidimensionales.
- **Modelos no relacionales y de grafos (NoSQL):** Consolidados a partir de 2010 para resolver cuellos de botella de escalabilidad horizontal y rigidez de esquemas. Reemplazan la normalización formal por esquemas flexibles de lectura (*schema-on-read*) divididos en cuatro familias: documentales (JSON/BSON), clave-valor, familias de columnas (*wide-column*) y grafos de propiedades, estos últimos formalizados mediante el estándar internacional ISO/IEC 39075:2024 (GQL).
- **Evolución contemporánea (NewSQL, multimodelo y bases de datos vectoriales):** Desde 2012, el modelo relacional tradicional derivó hacia sistemas distribuidos elásticos con consistencia transaccional estricta mediante algoritmos de consenso (NewSQL). De forma paralela, surgieron *motores convergentes multimodelo* y *bases de datos vectoriales* optimizadas para búsqueda de similitud semántica mediante grafos de mundo pequeño (*HNSW*) sobre representaciones densas de alta dimensionalidad (*embeddings*).

En la Figure 5 se resume la trayectoria evolutiva de estos paradigmas, destacando la bifurcación entre sistemas orientados a transacciones ACID y soluciones diseñadas para escalabilidad distribuida o cómputo multidimensional.

En la Table 3 se sistematizan los motores de bases de datos representativos de la disciplina, estructurados conforme a los paradigmas arquitectónicos presentados.

Table 3: Clasificación de sistemas gestores de bases de datos según su cronología formal.

Modelo arquitectónico	Época / Taxonomía	Sistemas y motores representativos	Mecanismo de estructuración predominante
Sistemas de archivos y métodos de acceso	Década de 1950 (Tipo 1)	IBM ISAM, VSAM, archivos secuenciales	Registros mapeados por <i>offset</i> físico en disco o cinta.
Modelo jerárquico	Década de 1960 (Tipo 1)	IBM IMS (<i>Information Management System</i>)	Estructuras en árbol con relaciones 1:N mediante punteros físicos padre-hijo.

Modelo arquitectónico	Época / Taxonomía	Sistemas y motores representativos	Mecanismo de estructuración predominante
Modelo en red	Década de 1960 (Tipo 1)	IDS, Cullinet IDMS (estándar CODASYL)	Grafos de registros con relaciones N:M gestionadas mediante conjuntos (<i>sets</i>) y apuntadores en anillo.
Modelo relacional transaccional (OLTP)	Década de 1970 (Tipo 2)	IBM System R, Ingres, IBM DB2, MySQL, SQLite	Tablas estructuradas por filas con tuplas y atributos atómicos, gobernadas por álgebra relacional y cumplimiento ACID.
Orientado a objetos (OODBMS)	Década de 1990 (Tipo 3)	ObjectStore, db4o, Versant, Objectivity/DB	Persistencia directa de instancias del lenguaje anfitrión con encapsulamiento, identidad propia (OID) y métodos.
Objeto-relacional (ORDBMS)	Década de 2000 (Tipo 2/3)	PostgreSQL (+ PostGIS), Oracle (v8i+), IBM Informix	Núcleo relacional con soporte nativo de tipos abstractos (ADT), polimorfismo, herencia de tablas e índices extensibles (GiST/SP-GiST).
NoSQL: Documental	Década de 2010	MongoDB, Apache CouchDB	Registros semiestructurados jerárquicos (JSON, BSON) con esquema dinámico (<i>schema-on-read</i>).
NoSQL: Grafos de propiedades	Década de 2010	Neo4j, Amazon Neptune, Memgraph	Nodos, aristas y propiedades tipadas optimizados para recorridos de relaciones complejas sin operaciones de combinación (<i>joins</i>).

Modelo arquitectónico	Época / Taxonomía	Sistemas y motores representativos	Mecanismo de estructuración predominante
NoSQL: Clave-valor	Década de 2010	Redis, Amazon DynamoDB, Memcached	Almacén asociativo indexado por <i>hash</i> unidimensional de muy baja latencia en memoria o disco.
NoSQL: Familias de columnas (<i>wide-column</i>)	Década de 2010	Apache Cassandra, Apache HBase, ScyllaDB	Tablas multidimensionales dispersas y particionadas horizontalmente, optimizadas para escrituras masivas distribuidas.
NewSQL / SQL distribuido	Década de 2010 (2012+)	Google Cloud Spanner, CockroachDB, YugabyteDB, TiDB	Álgebra relacional estándar y transaccionalidad ACID global sobre clústeres elásticos mediante protocolos de consenso (Paxos/Raft).
Modelo relacional analítico columnar (OLAP)	Década de 2010 (2019+)	DuckDB, ClickHouse, MonetDB	Álgebra relacional y sintaxis SQL estándar implementada sobre almacenamiento columnar y ejecución vectorial en memoria/CPU (SIMD).
Multimodelo nativo	Década de 2010 (2015+)	ArangoDB, Microsoft Azure Cosmos DB	Motores concebidos de origen para procesar de forma unificada grafos, documentos, clave-valor y estructuras tabulares.

Modelo arquitectónico	Época / Taxonomía	Sistemas y motores representativos	Mecanismo de estructuración predominante
Bases de datos vectoriales	Década de 2020 (2020+)	Milvus, Qdrant, Pinecone, Weaviate, Chroma	Almacenamiento e indexación de <i>embeddings</i> multidimensionales mediante grafos HNSW o cuantización vectorial para cálculo de distancias métricas.

La dimensión espacial como catalizador del modelo objeto-relacional

La adopción de modelos de bases de datos en geomática estuvo condicionada por la complejidad intrínseca de los datos geográficos frente a los datos alfanuméricos tradicionales. Esta complejidad se manifiesta en cuatro dimensiones críticas:

- **Multidimensionalidad:** Las entidades espaciales poseen extensión geométrica en dos, tres o cuatro dimensiones (coordenadas X , Y , elevación Z , y valores de medición lineal M).
- **Topología y continuidad:** Los objetos en el territorio exhiben relaciones espaciales continuas (adyacencia, contención, solapamiento, conectividad) que no se reducen a operadores de comparación escalar ($=$, $<$, $>$).
- **Tipologías heterogéneas:** Coexisten entidades vectoriales discretas (puntos, líneas, polígonos) y campos continuos (matrices ráster y mallas irregulares).
- **Ineficiencia de árboles de búsqueda unidimensionales:** Los árboles B y B+ son incapaces de indexar eficazmente cajas envolventes mínimas (*Bounding Boxes*) o rangos multidimensionales.

Como se sintetiza en la Figure 6, la evolución arquitectónica respondió a la necesidad de gestionar simultáneamente atributos escalares simples y geometrías dotadas de comportamiento topológico.

El modelo relacional tradicional garantizaba integridad referencial, consistencia transaccional y optimización sobre atributos atómicos escalares, pero carecía de mecanismos nativos para operar geometrías complejas sin desarticularlas en colecciones de coordenadas normalizadas. Por su parte, los sistemas orientados a objetos (OODBMS) permitían modelar geometrías y operadores topológicos como métodos de clase, pero sacrificaban la madurez del álgebra relacional, el lenguaje declarativo SQL y la eficiencia de los optimizadores basados en costos.

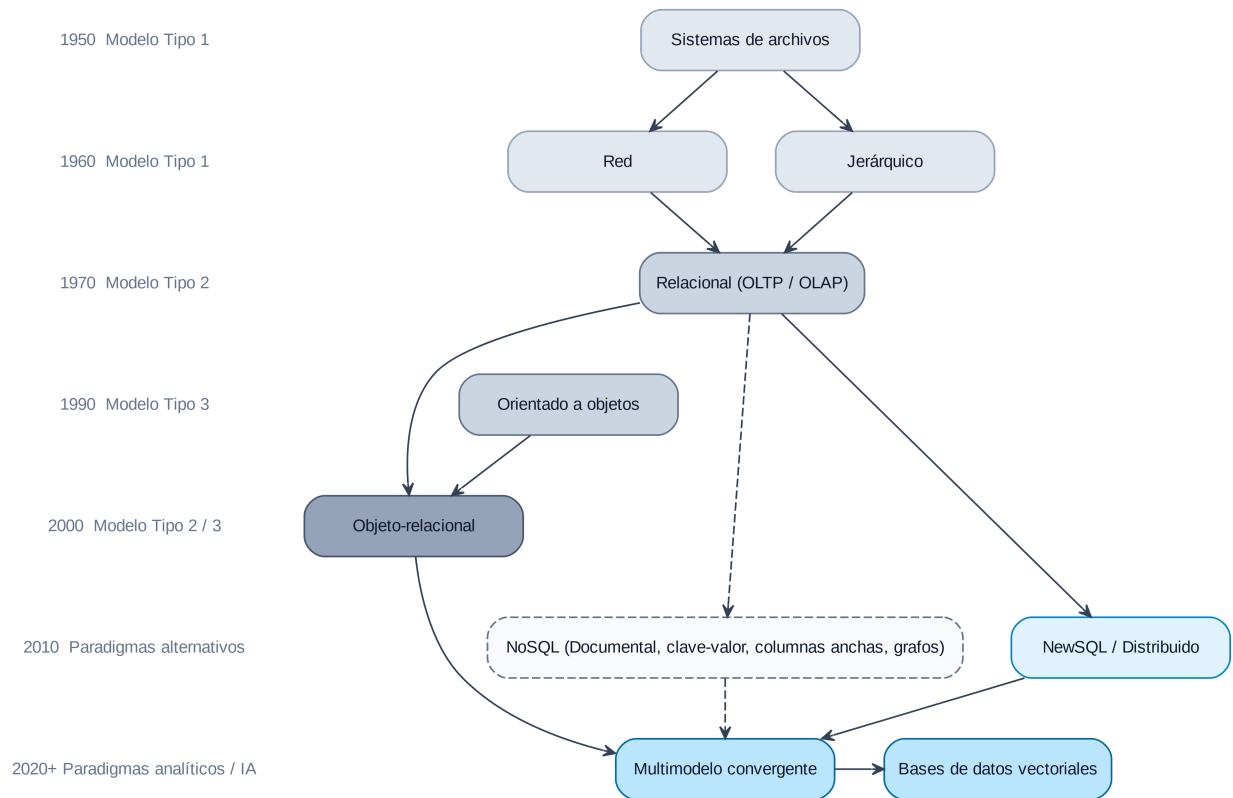


Figure 5: Cronología de modelos de bases de datos con alineación temporal formal.

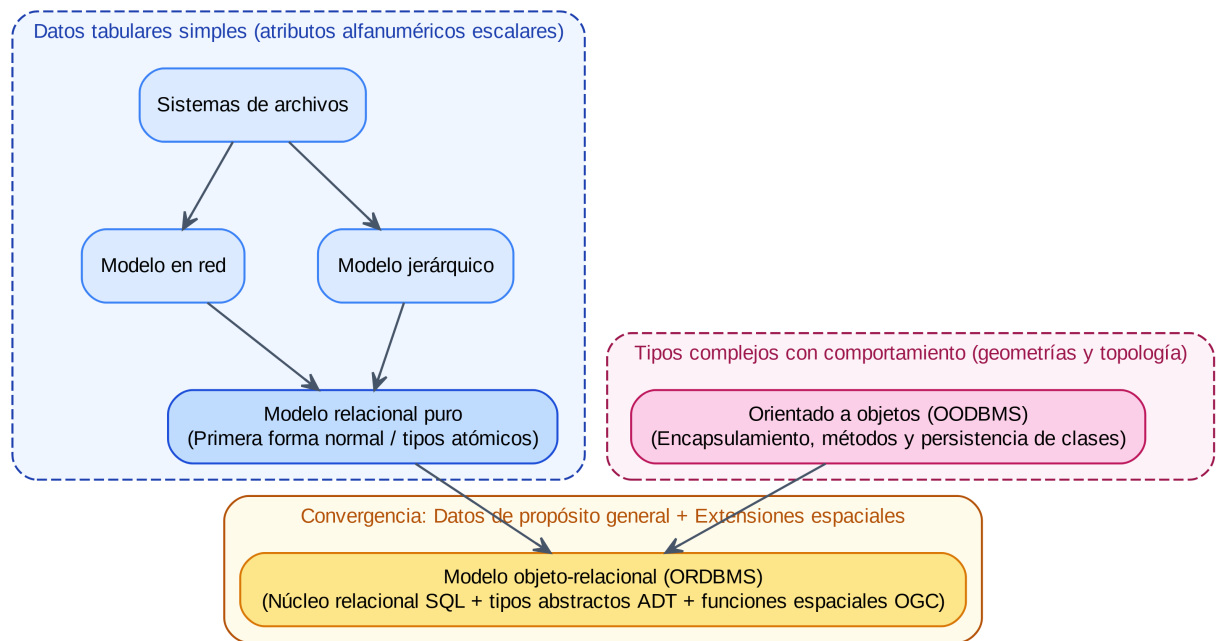


Figure 6: Convergencia de requerimientos de datos hacia el modelo objeto-relacional.

El modelo objeto-relacional resolvió esta encrucijada al extender el motor relacional con tipos de datos abstractos (ADT), índices espaciales multidimensionales (árboles R sobre infraestructura GiST) y predicados topológicos evaluables dentro del planificador de consultas.

Consideraciones arquitectónicas: extensibilidad y motores de cómputo analítico

Al analizar el ecosistema de datos espaciales y analíticos, es necesario distinguir entre la arquitectura de núcleo de un SGBD, los mecanismos de convergencia multimodelo por extensibilidad y los motores de procesamiento distribuido sobre lagos de datos:

- **Estatus arquitectónico de PostgreSQL y su convergencia multimodelo:** PostgreSQL es en su diseño de núcleo un sistema objeto-relacional (ORDBMS). La extensibilidad provista por su arquitectura permitió implementar soporte para datos semiestructurados (JSONB), tipos espaciales avanzados e índices multidimensionales mediante PostGIS, y búsqueda por similitud vectorial mediante `pgvector`. En sus versiones beta más recientes (como PostgreSQL 19), esta convergencia se amplía de forma nativa al incorporar consultas de grafos de propiedades mediante el estándar SQL/PgQ y capacidades avanzadas de optimización y asesoría de planes (`pg_plan_advice`). De este modo, aunque su fundamento teórico no es multimodelo nativo, opera en la práctica como un sustrato convergente altamente versátil para cargas de trabajo heterogéneas.

- **Diferenciación funcional de DuckDB:** DuckDB no plantea un nuevo modelo algebraico de datos, sino una reconfiguración de la arquitectura de ejecución. Se clasifica dentro del modelo relacional (Tipo 2), pero desacopla el paradigma tradicional de almacenamiento orientado a filas (*row-oriented*) para utilizar almacenamiento columnar vectorial en memoria. Su modelo de despliegue embebido (*in-process*) elimina la sobrecarga de red del modelo cliente-servidor, convirtiéndose en un motor OLAP especializado para análisis local sobre grandes volúmenes de datos tabulares y geoespaciales.
- **Distinción entre SGBD y motores de cómputo analítico (Apache Sedona):** Soluciones como Apache Sedona (anteriormente GeoSpark) no constituyen sistemas gestores de bases de datos (SGBD). Carecen de motores de almacenamiento persistente propios, gestores de transacciones ACID, subsistemas de bitácora (*Write-Ahead Logging*) y gestores de búfer en disco. Sedona es un **motor de procesamiento espacial distribuido** que actúa como capa analítica sobre entornos de cómputo en clúster (Apache Spark y Apache Flink), permitiendo evaluar predicados espaciales e indexación en memoria sobre archivos externos (GeoParquet, Shapefiles o capas de lagos de datos).

Arquitecturas para aplicaciones de sistemas de información geográfica

Históricamente, la vinculación entre los sistemas de información geográfica (SIG) y los sistemas de gestión de datos ha transitado por tres generaciones arquitectónicas consolidadas dentro del paradigma relacional y objeto-relacional (ORDBMS). Esta taxonomía describe la evolución del acoplamiento entre la geometría, los atributos descriptivos y el motor de ejecución en entornos transaccionales operacionales (OLTP). Si bien los paradigmas analíticos contemporáneos (como motores columnares embebidos, cómputo distribuido sobre clústeres, o almacenes orientados a grafos y NoSQL) responden a demandas analíticas de frontera que se abordan en los proyectos de investigación al cierre del curso, el modelo integrado objeto-relacional constituye la base conceptual y operativa sobre la cual se fundamenta la integridad, concurrencia y estandarización de los datos geoespaciales corporativos.

En la [Figure 7](#) se ilustra esta evolución conceptual desde esquemas propietarios aislados hasta la centralización transaccional en motores espaciales integrados.

Arquitectura ad-hoc

En esta configuración inicial, los paquetes de software SIG implementan formatos binarios cerrados y algoritmos propietarios de cálculo geométrico ejecutados directamente sobre el sistema de archivos:

- **Ventajas:** Alto rendimiento y eficiencia computacional para operaciones cartográficas locales en estaciones monousuario, al carecer de sobrecarga (*overhead*) por capas de red o bloqueos transaccionales.

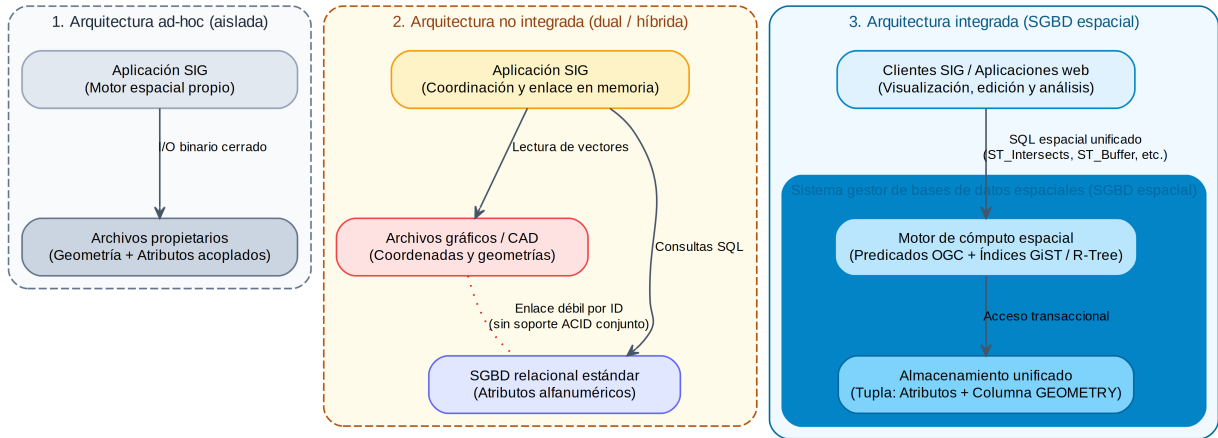


Figure 7: Evolución de las arquitecturas de integración entre SIG y bases de datos.

- **Limitaciones:** Nula extensibilidad hacia nuevas aplicaciones, imposibilidad de compartir datos fuera del entorno de origen, ausencia de control de concurrencia y proliferación de información redundante e inconsistente.

Arquitectura no integrada (dual o híbrida)

Este esquema desacopla el almacenamiento en dos repositorios independientes: los atributos temáticos se delegan a un SGBD relacional estándar, mientras que las geometrías continúan gestionándose mediante el sistema de archivos tradicional:

- **Ventajas:** Incorpora las capacidades analíticas, de seguridad y de consulta declarativa del SGBD para la fracción alfanumérica de la información.
- **Limitaciones:** La responsabilidad de integrar y sincronizar la geometría con la tabla recae exclusivamente en la capa de aplicación (*application programs*), lo que genera un rendimiento inferior al de las soluciones ad-hoc, sobrecarga de memoria en el cliente y riesgo permanente de inconsistencia transaccional (huérfanos espaciales o tablas sin geometría asociada).

Arquitectura integrada (SGBD espacial)

Representa el estándar contemporáneo, donde el sistema gestor de bases de datos asume tanto la geometría como los atributos descriptivos dentro de un mismo motor estructurado:

- **Reutilización y extensibilidad:** Desacopla los datos espaciales de los programas SIG clientes, permitiendo que múltiples aplicaciones heterogéneas (escritorio, servicios web

OGC, pipelines en Python/R/Julia) consuman y actualicen las mismas capas de forma concurrente.

- **Procesamiento en el servidor:** Las funciones analíticas, el cálculo de relaciones topológicas y la indexación multidimensional se ejecutan directamente en el motor de la base de datos, optimizando el ancho de banda al retornar únicamente el subconjunto filtrado.
- **Compromiso de rendimiento (*trade-off*):** Aunque puede presentar un costo operativo marginalmente superior frente a algoritmos locales ad-hoc debido a la gestión de transacciones ACID y la serialización SQL, dicho margen queda plenamente compensado por la integridad estructural, la escalabilidad y la interoperabilidad institucional de los datos.

Distribución de responsabilidades entre el SIG y el SGBD objeto-relacional

La adopción de una arquitectura integrada formaliza un esquema multicapa donde se dividen estrictamente las tareas entre la aplicación cliente y el motor gestor de bases de datos, tal como se esquematiza en la Figure 8.

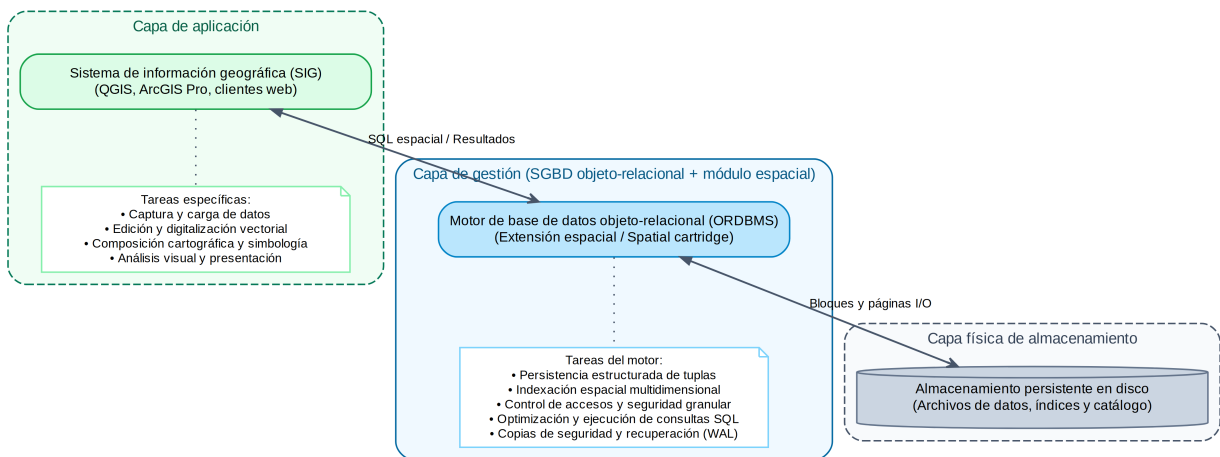


Figure 8: Distribución de capas y responsabilidades entre el software SIG y el SGBD objeto-relacional.

Bajo este modelo de distribución operativa se derivan ventajas determinantes:

- **Descarga computacional de la aplicación cliente:** Al incorporar un **módulo espacial** (*spatial extension* o *cartridge*) directamente acoplado al núcleo del motor, los algoritmos geométricos computacionalmente intensivos (cálculo de áreas, uniones espaciales, generación de zonas de influencia o evaluación de relaciones topológicas) se procesan dentro del servidor.

- **Unificación de datos convencionales y espaciales:** El SGBD administra atributos alfanuméricos y objetos geométricos bajo un mismo esquema declarativo, garantizando que las reglas de integridad referencial, directivas de seguridad y restricciones de dominio apliquen uniformemente sobre toda la entidad.
- **Simplificación del ciclo de vida del software:** Las aplicaciones SIG se liberan de la responsabilidad de implementar motores de indexación propios o controladores de concurrencia a nivel de archivo. Esto reduce la complejidad del código cliente y previene inconsistencias por accesos simultáneos.

PostgreSQL y la extensión PostGIS

PostgreSQL es un sistema gestor de bases de datos objeto-relacional (ORDBMS) de código abierto, diseñado con una arquitectura extensible que permite incorporar nuevos tipos de datos, funciones definidas por el usuario en múltiples lenguajes (C, PL/pgSQL, Python) y métodos de acceso indexados especializados.

PostGIS es la extensión espacial oficial de PostgreSQL. Transforma el motor relacional en un SGBD espacial completo al incorporar:

- Tipos de datos espaciales geométricos vectoriales (**geometry**), geodésicos (**geography**), matriciales (**raster**) y estructuras topológicas (**topology**).
- Funciones de análisis espacial conformes a las especificaciones internacionales de la OGC.
- Soporte para métodos de indexación espacial mediante árboles de búsqueda generalizada (GiST, *Generalized Search Tree*) y árboles R.

i Instrucciones específicas para el sistema anfitrión (Windows)

Cuando se administra PostgreSQL de forma nativa sobre el sistema anfitrión Windows, el servicio del clúster puede gestionarse desde la consola de servicios (**services.msc**), identificando la entrada **postgresql-x64-<version>**. Alternativamente, desde una terminal de PowerShell con privilegios administrativos, se puede consultar el estado del servicio mediante:

```
Get-Service -Name postgresql*
```

Para asegurar que las utilidades de línea de comandos (**psql**, **pg_dump**, **pg_restore**) respondan globalmente en la consola del sistema anfitrión, la ruta binaria del motor (por ejemplo, **C:\Program Files\PostgreSQL\<version>\bin**) debe estar registrada en la variable de entorno del sistema **Path**.

Verificación de PostGIS en el sistema anfitrión (Windows)

Para comprobar que el entorno espacial se encuentra habilitado y operativo en la base de datos de trabajo `sig_db_unal`, se consulta la firma completa de dependencias compiladas contra el motor relacional:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

# Inspección de versión y librerías geoespaciales vinculadas a PostGIS
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
  -x -P format=wrapped -P columns=90 \
  -c "SELECT postgis_full_version();"

-[ RECORD 1 ]-----+-----
postgis_full_version | POSTGIS="3.6.2 3.6.2" [EXTENSION] PGSQL="180" GEOS="3.14.1dev-CAPI.
                        | .-1.20.4" PROJ="8.2.1 NETWORK_ENABLED=OFF URL_ENDPOINT=https://cdn..
                        | .proj.org USER_WRITABLE_DIRECTORY=C:\WINDOWS\ServiceProfiles\Networ.
                        | .kService\AppData\Local\proj DATABASE_PATH=D:\Programs\PostgreSQL\1.
                        | .8\share\contrib\postgis-3.6\proj\proj.db" (compiled against PROJ 8.
                        | ..2.1) LIBXML="2.12.5" LIBJSON="0.12" LIBPROTOBUF="1.2.1" WAGYU="0..
                        | .5.0 (Internal)"
```

El resultado de esta función expone las librerías base sobre las cuales opera el motor espacial:

- **GEOS (*Geometry Engine Open Source*)**: Provee las primitivas de cálculo computacional y evaluación de predicados topológicos.
- **PROJ**: Motor de transformación y reproyección entre sistemas de referencia de coordenadas.
- **GDAL (*Geospatial Data Abstraction Library*)**: Librería de traducción y soporte para formatos ráster y vectoriales.
- **LIBXML2**: Procesador de serializaciones XML/GML.

Estándar OGC Simple Features for SQL (ISO 19125)

El Open Geospatial Consortium (OGC) y la Organización Internacional de Normalización (ISO/TC 211) formularon la especificación *Simple Feature Access* (SFA) —conocida comúnmente en entornos relacionales como *Simple Features for SQL* (SFS)—, estandarizada bajo la norma ISO 19125. Esta especificación se estructura formalmente en dos partes complementarias: la Parte 1 (*Common Architecture*), que define el modelo conceptual de geometrías, la jerarquía de tipos (`Point`, `Curve`, `Surface`, `GeometryCollection`) y su vinculación con sistemas de referencia de coordenadas [`@ogc_sfa_part1_2011`]; y la Parte 2 (*SQL Option*), que

especifica la implementación técnica, el esquema relacional y las funciones en SQL requeridas para almacenar, manipular y consultar dichas colecciones geométricas en motores de bases de datos relacionales [ogc_sfa_part2_2010].

Jerarquía de tipos geométricos OGC

El estándar define una taxonomía orientada a objetos en la que la clase abstracta raíz es **Geometry**. En la Figure 9 se presenta el diagrama de clases conforme a la especificación formal ISO 19125 / OGC SFA.

Para la interpretación de la Figure 9 se deben considerar las siguientes convenciones de la notación UML:

- **Clases abstractas («Abstract»):** Entidades conceptuales que no admiten instanciación directa (**Geometry**, **Curve**, **Surface**, **MultiSurface**, **MultiCurve**). Definen contratos de métodos y propiedades que heredan las clases concretas.
- **Generalización ($\triangle$):** Relación de herencia (*es-un*). La flecha orientada hacia la superclase indica que la subclase adquiere sus atributos, operaciones y restricciones topológicas.
- **Agregación ($\diamond$):** Relación estructural (*compuesto-por*). El rombo ubicado en el extremo de la clase contenedora indica que esta se compone de una colección de elementos de la clase vinculada.
- **Multiplicidad:** Restricción cardinal de las asociaciones; por ejemplo, $2..*$ denota la obligatoriedad de al menos dos vértices para instanciar un **LineString**, mientras que $1..*$ exige al menos un anillo delimitador para un **Polygon**.

Representación espacial de primitivas y colecciones

Cada entidad definida en el estándar posee una signature geométrica y una dimensión topológica intrínseca. En la Figure 10 se presenta la morfología de las geometrías fundamentales y sus agrupaciones.

En la Table 4 se detallan las características topológicas y dimensionales de los tipos definidos en la norma.

Table 4: Primitivas y colecciones geométricas del estándar OGC.

Tipo geométrico	Dimensión topológica	Descripción formal OGC
Point	0	Posición geométrica aislada descrita por un único par ordenado (X, Y) .
LineString	1	Curva lineal interpolada entre dos o más vértices.

Tipo geométrico	Dimensión topológica	Descripción formal OGC
LinearRing	1	LineString cerrada y no autosecante (define límites poligonales).
Polygon	2	Superficie planar delimitada por un anillo exterior y cero o más interiores.
MultiPoint	0	Agrupación no conexas de puntos independientes.
MultiLineString	1	Conjunto estructurado de cadenas de líneas.
MultiPolygon	2	Conjunto conexo o disjunto de polígonos.
GeometryCollection	Variable	Agrupación heterogénea arbitraria de entidades geométricas.

Serialización y formatos estándar: WKT y WKB

Para garantizar el intercambio de geometrías entre sistemas heterogéneos, el estándar OGC define dos formatos de interoperabilidad:

Formato Well-Known Text (WKT)

WKT es una codificación en texto plano legible por humanos que define la estructura del tipo geométrico y sus secuencias de coordenadas:

- POINT(1000000 1000000)
- LINESTRING(1000000 1000000, 1050000 1050000, 1100000 1020000)
- POLYGON((1000000 1000000, 1100000 1000000, 1100000 1100000, 1000000 1100000, 1000000 1000000))

En superficies poligonales, el primer anillo define el perímetro exterior, mientras que los anillos sucesivos delimitan perforaciones interiores:

```
POLYGON((0 0, 10 0, 10 10, 0 10, 0 0), (2 2, 4 2, 4 4, 2 4, 2 2))
```

PostGIS extiende el estándar WKT incorporando componentes tridimensionales y de medición lineal: POINT Z, POINT M y POINT ZM.

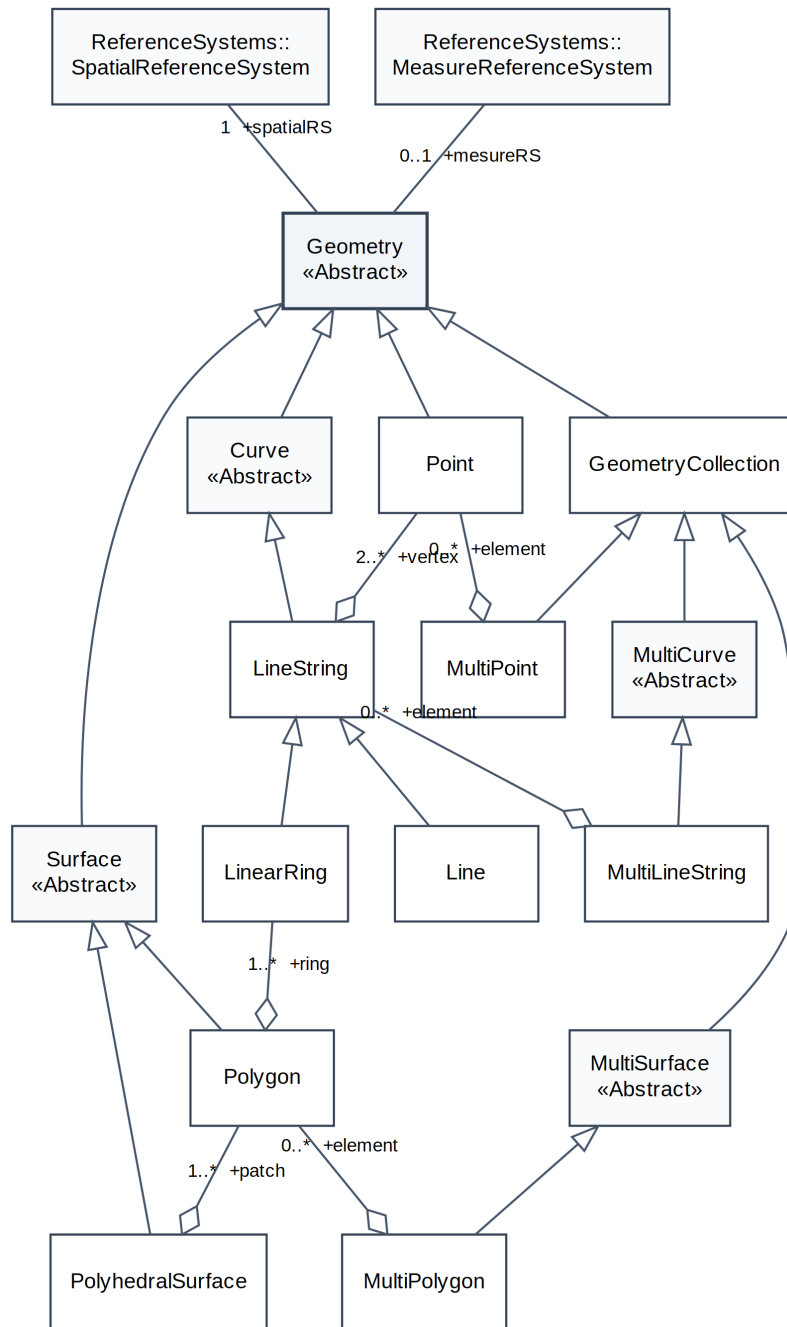


Figure 9: Diagrama de clases UML para la jerarquía de geometrías según ISO 19125 / OGC Simple Feature Access.

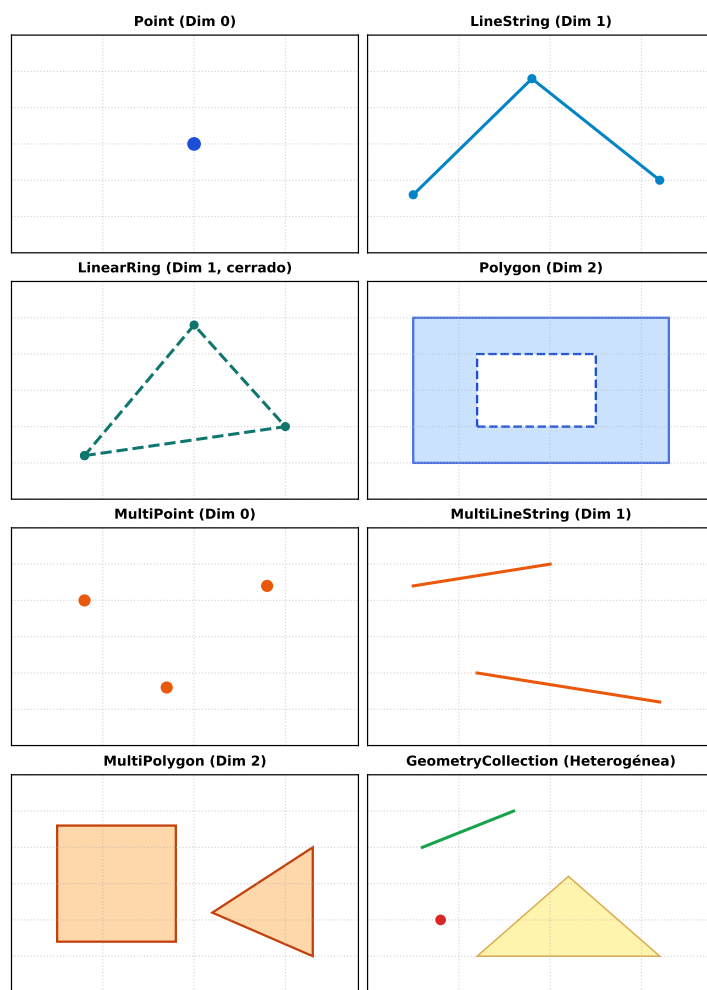


Figure 10: Instanciación espacial y morfología de tipos geométricos según el estándar OGC.

Formato Well-Known Binary (WKB) y EWKB

WKB es una representación binaria estructurada en secuencias de bytes (*byte arrays*), optimizada para la transferencia en red y el almacenamiento interno en páginas de disco:

- Incorpora indicadores de ordenamiento de bytes (*Endianness*: Big-Endian o Little-Endian).
- Define identificadores numéricos que codifican el tipo de geometría.
- Preserva la precisión numérica en coma flotante de doble precisión (IEEE 754) para cada coordenada.

PostGIS implementa el formato extendido **EWKB** (*Extended Well-Known Binary*), el cual embebe directamente el identificador del sistema de referencia espacial (SRID) en la cabecera binaria del objeto geométrico.

Representación espacial de primitivas y colecciones en PostGIS

Para verificar la persistencia física de las geometrías OGC y evaluar el intercambio de datos mediante los formatos canónicos WKT y WKB directamente en el motor relacional del servidor local, el flujo de trabajo se estructura en dos fases complementarias:

1. **Persistencia relacional en PostGIS:** Se provisiona una tabla polimórfica (`public.muestras_geometrias_ogc`) con una columna geométrica sin modificadores de tipo restrictivos (*typmod*), permitiendo almacenar las ocho primitivas y colecciones fundamentales mediante constructores estándar `ST_GeomFromText`. Conforme a la gramática formal de la norma ISO 19125-2, el tipo `LinearRing` carece de un constructor textual WKT autónomo en el estándar SQL; por consiguiente, se persiste como una curva `LineString` cerrada con coincidencia topológica estricta entre el vértice inicial y final.
2. **Extracción binaria y renderizado en Python:** El entorno ejecuta una consulta relacional solicitando la codificación canónica *Well-Known Binary* (`ST_AsBinary`), decodifica los flujos de bytes mediante `shapely.wkb` y genera la matriz gráfica comparativa.

El siguiente bloque de instrucciones DDL y DML ejecuta el aprovisionamiento y poblamiento de la relación en el clúster PostgreSQL:

- **Eliminación condicional previa:** Suprime la tabla `public.muestras_geometrias_ogc` si existe en el esquema (`DROP TABLE IF EXISTS`), garantizando la idempotencia del script ante múltiples ejecuciones.
- **Definición del esquema de la tabla:** Crea la relación especificando una clave primaria serial, identificadores de ordenamiento, atributos descriptivos y una columna espacial de tipo genérico `geometry` sin restricción de subtipo (*typmod*).

- **Inserción de atributos y geometrías:** Registra los datos alfanuméricos e instancia físicamente cada objeto geométrico a partir de literales WKT mediante la función constructora `ST_GeomFromText`.

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_muestras="
SET client_min_messages = warning;
DROP TABLE IF EXISTS public.muestras_geometrias_ogc;
RESET client_min_messages;

CREATE TABLE public.muestras_geometrias_ogc (
    id serial PRIMARY KEY,
    orden integer NOT NULL,
    nombre varchar(30) NOT NULL,
    descripcion varchar(50) NOT NULL,
    geom geometry NOT NULL
);

INSERT INTO public.muestras_geometrias_ogc (orden, nombre, descripcion, geom) VALUES
(1, 'Point', 'Point (Dim 0)',
    ST_GeomFromText('POINT(2 1.5)'),
(2, 'LineString', 'LineString (Dim 1)',
    ST_GeomFromText('LINESTRING(0.5 0.8, 1.8 2.4, 3.2 1.0)'),
(3, 'LinearRing', 'LinearRing (Dim 1, cerrado)',
    ST_GeomFromText('LINESTRING(0.8 0.6, 3.0 1.0, 2.0 2.4, 0.8 0.6)'),
(4, 'Polygon', 'Polygon (Dim 2)',
    ST_GeomFromText('POLYGON((0.5 0.5, 3.3 0.5, 3.3 2.5, 0.5 2.5, 0.5 0.5), (1.2 1.0, 2.5 1.0, 2.5 1.0, 1.2 1.0)'),
(5, 'MultiPoint', 'MultiPoint (Dim 0)',
    ST_GeomFromText('MULTIPOINT(0.8 2.0, 1.7 0.8, 2.8 2.2)'),
(6, 'MultiLineString', 'MultiLineString (Dim 1)',
    ST_GeomFromText('MULTILINESTRING((0.5 2.2, 2.0 2.5), (1.2 1.0, 3.2 0.6)'),
(7, 'MultiPolygon', 'MultiPolygon (Dim 2)',
    ST_GeomFromText('MULTIPOLYGON(((0.5 0.7, 1.8 0.7, 1.8 2.3, 0.5 2.3, 0.5 0.7)), ((2.2 1.1, 3.2 1.1, 3.2 0.5, 2.2 0.5, 2.2 1.1))),
(8, 'GeometryCollection', 'GeometryCollection (Heterogénea)',
    ST_GeomFromText('GEOMETRYCOLLECTION(POLYGON((1.2 0.5, 3.2 0.5, 2.2 1.6, 1.2 0.5)), LINESTRING(0.5 0.8, 1.8 2.4, 3.2 1.0))'),
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-c "${query_muestras}"
```

SET

```

DROP TABLE
RESET
CREATE TABLE
INSERT 0 8

```

Una vez confirmada la inserción en el motor, el bloque subsecuente recupera las geometrías en WKB, reconstruye los objetos topológicos e ilustra la morfología de cada tipo en la [?@fig-geometrias-graficas-postgis](#):

Python

```

# =====
# 0. Módulos y bibliotecas requeridas
# =====
# 'os': Módulo estándar para interactuar con el sistema operativo;
# permite recuperar variables de entorno del sistema (credenciales seguras).
import os

# 'psycopg2': Controlador canónico (driver) de PostgreSQL para Python;
# gestiona el protocolo de red, la conexión y la ejecución de sentencias SQL.
import psycopg2

# 'matplotlib.pyplot': Biblioteca estándar de graficación técnica y científica;
# provee las funciones para construir y configurar lienzo vectoriales 2D.
import matplotlib.pyplot as plt

# 'shapely.wkb': Submódulo de la biblioteca geoespacial Shapely (basada en GEOS);
# permite deserializar cadenas binarias en formato Well-Known Binary (WKB)
# y reconstruirlas como objetos geométricos espaciales manipulables en memoria.
from shapely import wkb

# -----
# 1. Conexión a la base de datos relacional
# -----
conn = psycopg2.connect(
    host=os.environ.get("LOCAL_PGHOST"),
    port=os.environ.get("LOCAL_PGPORT"),
    dbname=os.environ.get("LOCAL_PGDATABASE"),
    user=os.environ.get("LOCAL_PGUSER"),
    password=os.environ.get("LOCAL_PGPASSWORD")
)

```

```

# -----
# 2. Extracción de entidades en formato binario estándar (WKB)
# -----
cursor = conn.cursor()
cursor.execute("""
    SELECT orden, nombre, descripcion, ST_AsBinary(geom) AS wkb
    FROM public.muestras_geometrias_ogc
    ORDER BY orden ASC;
""")
registros = cursor.fetchall()
cursor.close()
conn.close()

# -----
# 3. Configuración del lienzo gráfico (Matplotlib)
# -----
fig, axes = plt.subplots(4, 2, figsize=(6.5, 9.0), constrained_layout=True)
axes_flat = axes.flatten()

# -----
# 4. Función de despacho polimórfico para renderizado geométrico
# -----
def graficar_geometria(ax, geom, nombre):
    if geom.geom_type == 'Point':
        ax.plot(geom.x, geom.y, 'o', color='#1d4ed8', markersize=8)

    elif geom.geom_type == 'LineString':
        x, y = geom.xy
        if nombre == 'LinearRing':
            ax.plot(x, y, '--o', color='#0f766e', linewidth=2, markersize=5)
        else:
            ax.plot(x, y, '-o', color='#0284c7', linewidth=2, markersize=5)

    elif geom.geom_type == 'Polygon':
        ax.fill(*geom.exterior.xy, color='#bfd8fe', alpha=0.8, edgecolor='#1d4ed8', linewidth=1)
        for interior in geom.interiors:
            ax.fill(*interior.xy, color='ffffff', edgecolor='#1d4ed8', linewidth=1.5, linestyle='dashed')

    elif geom.geom_type == 'MultiPoint':
        for pt in geom.geoms:
            ax.plot(pt.x, pt.y, 'o', color='#ea580c', markersize=7)

```

```

elif geom.geom_type == 'MultiLineString':
    for line in geom.geoms:
        ax.plot(*line.xy, '-', color='#ea580c', linewidth=2)

elif geom.geom_type == 'MultiPolygon':
    for poly in geom.geoms:
        ax.fill(*poly.exterior.xy, color='#fed7aa', edgecolor='#c2410c', linewidth=1.5)

elif geom.geom_type == 'GeometryCollection':
    for part in geom.geoms:
        if part.geom_type == 'Polygon':
            ax.fill(*part.exterior.xy, color='#fef08a', edgecolor='#ca8a04', alpha=0.7)
        elif part.geom_type == 'LineString':
            ax.plot(*part.xy, '-', color='#16a34a', linewidth=2)
        elif part.geom_type == 'Point':
            ax.plot(part.x, part.y, 'o', color='#dc2626', markersize=6)

# -----
# 5. Iteración, deserialización binaria y graficación
# -----
for idx, (orden, nombre, descripcion, binario_wkb) in enumerate(registros):
    ax = axes_flat[idx]
    geometria = wkb.loads(bytes(binario_wkb))
    graficar_geometria(ax, geometria, nombre)
    ax.set_title(descripcion, fontsize=9, weight='bold')
    _ = ax.set_xlim(0, 3.8)
    _ = ax.set_ylim(0, 3.0)
    ax.grid(True, linestyle=':', alpha=0.5)
    ax.tick_params(left=False, bottom=False, labelleft=False, labelbottom=False)

plt.show()

```

R

```

# =====
# 0. Módulos y bibliotecas requeridas
# =====
# 'DBI': Interfaz unificada de conexión a bases de datos relacionales en R.
library(DBI)

```

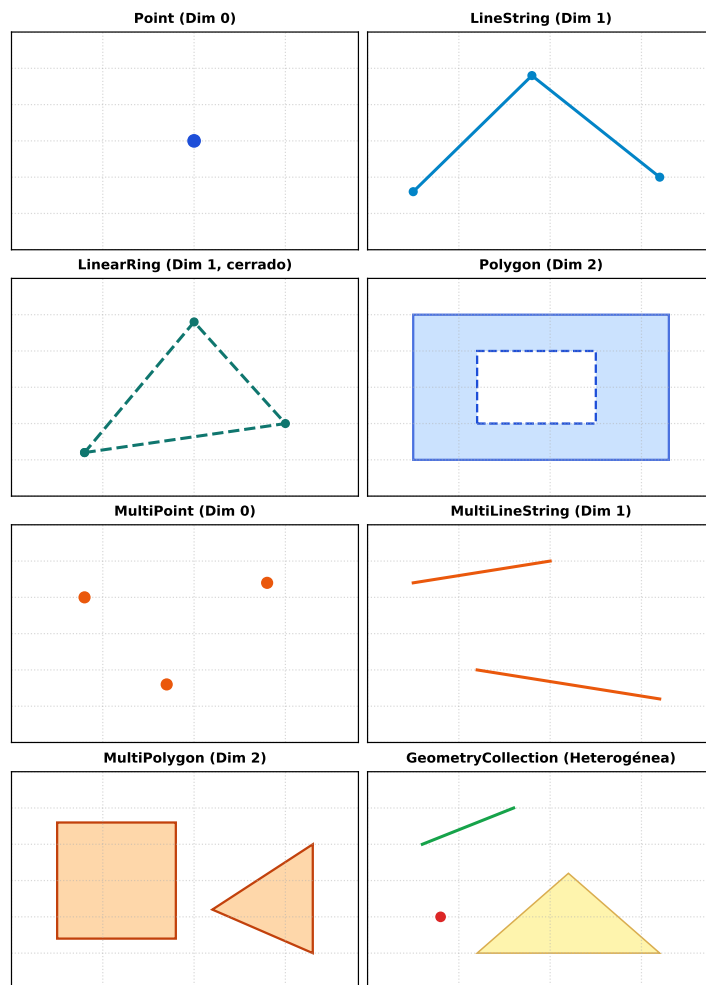


Figure 11: Instanciación espacial y morfología de tipos geométricos según el estándar OGC en Python.

```

# 'RPostgres': Controlador optimizado en C++ para comunicación nativa con PostgreSQL.
library(RPostgres)

# 'sf': Biblioteca estándar para geometrías vectoriales bajo Simple Features;
# enlaza con GDAL, GEOS y PROJ, proveyendo deserialización nativa de WKB.
library(sf)

# -----
# 1. Conexión a la base de datos relacional
# -----
conn <- dbConnect(
  RPostgres::Postgres(),
  host = Sys.getenv("LOCAL_PGHOST"),
  port = as.integer(Sys.getenv("LOCAL_PGPORT")),
  dbname = Sys.getenv("LOCAL_PGDATABASE"),
  user = Sys.getenv("LOCAL_PGUSER"),
  password = Sys.getenv("LOCAL_PGPASSWORD")
)

# -----
# 2. Extracción de entidades en formato binario estándar (WKB)
# -----
query <- "
  SELECT orden, nombre, descripcion, ST_AsBinary(geom) AS wkb
  FROM public.muestras_geometrias_ogc
  ORDER BY orden ASC;
"
registros <- dbGetQuery(conn, query)
dbDisconnect(conn)

# Conversión del vector binario WKB a objetos de clase 'sfc' (Simple Feature Column)
class(registros$wkb) <- "WKB"
geometrias <- st_as_sfc(registros$wkb, EWKB = FALSE)

# -----
# 3. Configuración del lienzo gráfico (Base R Graphics)
# -----
# Márgenes calibrados por celda (inferior, izquierdo, superior, derecho)
par(mfrow = c(4, 2), mar = c(1.8, 1.8, 2.2, 1.2), oma = c(0, 0, 0, 0))

# -----
# 4. Función de despacho polimórfico para renderizado geométrico

```

```

# -----
graficar_geometria_r <- function(geom, nombre, descripcion) {
  # Inicialización del marco cartesiano libre de distorsión por relación de aspecto fija
  plot.new()
  plot.window(xlim = c(0, 3.8), ylim = c(0, 3.0))

  # Cuadrícula homogénea de 4 columnas x 3 filas con estilo punteado
  grid(nx = 4, ny = 3, lty = 3, col = "gray75")
  box(col = "#334155", lwd = 1.0)
  title(main = descripcion, cex.main = 1.05, font.main = 2, line = 0.6)

  # Renderizado condicional según el tipo primitivo
  if (inherits(geom, "POINT")) {
    points(geom[1], geom[2], pch = 19, col = "#1d4ed8", cex = 2.2)
  } else if (inherits(geom, "LINESTRING")) {
    if (nombre == "LinearRing") {
      lines(geom[, 1], geom[, 2], col = "#0f766e", lty = 2, lwd = 2.5)
      points(geom[, 1], geom[, 2], pch = 19, col = "#0f766e", cex = 1.2)
    } else {
      lines(geom[, 1], geom[, 2], col = "#0284c7", lwd = 2.5)
      points(geom[, 1], geom[, 2], pch = 19, col = "#0284c7", cex = 1.2)
    }
  }

  } else if (inherits(geom, "POLYGON")) {
    # Anillo exterior
    polygon(geom[[1]][, 1], geom[[1]][, 2], col = "#bfdbfe", border = "#1d4ed8", lwd = 2.0)
    # Huecos interiores (islas)
    if (length(geom) > 1) {
      for (h in 2:length(geom)) {
        polygon(geom[[h]][, 1], geom[[h]][, 2], col = "white", border = "#1d4ed8", lty = 2, lwd = 2.0)
      }
    }
  }

  } else if (inherits(geom, "MULTIPOINT")) {
    points(geom[, 1], geom[, 2], pch = 19, col = "#ea580c", cex = 1.8)
  }

  } else if (inherits(geom, "MULTILINESTRING")) {
    for (linea in geom) {
      lines(linea[, 1], linea[, 2], col = "#ea580c", lwd = 2.5)
    }
  }
}

```

```

} else if (inherits(geom, "MULTIPOLYGON")) {
  for (poligono in geom) {
    polygon(poligono[[1]][, 1], poligono[[1]][, 2], col = "#fed7aa", border = "#c2410c", lwd = 2)
  }
}

} else if (inherits(geom, "GEOMETRYCOLLECTION")) {
  for (elem in geom) {
    if (inherits(elem, "POLYGON")) {
      polygon(elem[[1]][, 1], elem[[1]][, 2], col = "#fef08a", border = "#ca8a04", lwd = 2)
    } else if (inherits(elem, "LINESTRING")) {
      lines(elem[, 1], elem[, 2], col = "#16a34a", lwd = 2.5)
    } else if (inherits(elem, "POINT")) {
      points(elem[1], elem[2], pch = 19, col = "#dc2626", cex = 1.8)
    }
  }
}
}

# -----
# 5. Iteración y graficación
# -----
for (i in seq_len(nrow(registros))) {
  graficar_geometria_r(geometrias[[i]], registros$nombre[i], registros$descripcion[i])
}

```

Linking to GEOS 3.12.1, GDAL 3.8.4, PROJ 9.4.0; sf_use_s2() is TRUE

Julia

Starting Julia ...

```

julia>
ENV["GKSwstype"] = "100"
"100"

```

```

julia>
using LibPQ

```

```

julia>
using DataFrames

```



Figure 12: Instanciación espacial y morfología de tipos geométricos según el estándar OGC en R.

```

julia>
using Plots

julia>
gr()
Plots.GRBackend()

julia>
try
    host = get(ENV, "LOCAL_PGHOST", "db-postgis")
    port = get(ENV, "LOCAL_PGPORT", "5432")
    db = get(ENV, "LOCAL_PGDATABASE", "sig_db_unal")
    user = get(ENV, "LOCAL_PGUSER", "profe_unal")
    pass = get(ENV, "LOCAL_PGPASSWORD", "geomatica2025")

    cadena_conexion = "host=$(host) port=$(port) dbname=$(db) user=$(user) password=$(pass
    )"
    canal = LibPQ.Connection(cadena_conexion)

    query = "SELECT orden, nombre, descripcion, ST_AsText(geom) AS wkt FROM public.muestra
    s_geometrias_ogc ORDER BY orden ASC;"
    df = DataFrame(execute(canal, query))
    close(canal)

    # Configuración base balanceada para cada celda
    cfg(tit) = (
        xlims = (0, 3.8),
        ylims = (0, 3.0),
        title = tit,
        titlefontsize = 11,
        framestyle = :box,
        grid = true,
        gridstyle = :dot,
        gridalpha = 0.5,
        xticks = (0.5:1.0:3.5, [ "", "", "", "" ]),
        yticks = (0.5:1.0:2.5, [ "", "", "" ]),
        legend = false,
        top_margin = 4Plots.mm,
        bottom_margin = 2Plots.mm
    )

    # 1. Point
    local p1 = plot(; cfg(df.descripcion[1])...)
    scatter!(p1, [2.0], [1.5], color = "#1d4ed8", markersize = 10)

    # 2. LineString
    local p2 = plot(; cfg(df.descripcion[2])...)
    plot!(p2, [0.5, 1.8, 3.2], [0.8, 2.4, 1.0], color = "#0284c7", linewidth = 2.5, marker

```

```

        = :circle, markersize = 6)

# 3. LinearRing
local p3 = plot(; cfg(df.descripcion[3])...)
plot!(p3, [0.8, 3.0, 2.0, 0.8], [0.6, 1.0, 2.4, 0.6], color = "#0f766e", linestyle = :
dash, linewidth = 2.5, marker = :circle, markersize = 6)

# 4. Polygon
local p4 = plot(; cfg(df.descripcion[4])...)
plot!(p4, Shape([0.5, 3.3, 3.3, 0.5], [0.5, 0.5, 2.5, 2.5]), fillcolor = "#bfdbf", fi
llalpha = 0.8, linecolor = "#1d4ed8", linewidth = 2)
plot!(p4, Shape([1.2, 2.5, 2.5, 1.2], [1.0, 1.0, 2.0, 2.0]), fillcolor = :white, linec
olor = "#1d4ed8", linestyle = :dash, linewidth = 2)

# 5. MultiPoint
local p5 = plot(; cfg(df.descripcion[5])...)
scatter!(p5, [0.8, 1.7, 2.8], [2.0, 0.8, 2.2], color = "#ea580c", markersize = 8)

# 6. MultiLineString
local p6 = plot(; cfg(df.descripcion[6])...)
plot!(p6, [0.5, 2.0], [2.2, 2.5], color = "#ea580c", linewidth = 2.5)
plot!(p6, [1.2, 3.2], [1.0, 0.6], color = "#ea580c", linewidth = 2.5)

# 7. MultiPolygon
local p7 = plot(; cfg(df.descripcion[7])...)
plot!(p7, Shape([0.5, 1.8, 1.8, 0.5], [0.7, 0.7, 2.3, 2.3]), fillcolor = "#fed7aa", li
necolor = "#c2410c", linewidth = 2)
plot!(p7, Shape([2.2, 3.3, 3.3, 2.2], [1.1, 0.5, 2.0, 1.1]), fillcolor = "#fed7aa", li
necolor = "#c2410c", linewidth = 2)

# 8. GeometryCollection
local p8 = plot(; cfg(df.descripcion[8])...)
plot!(p8, Shape([1.2, 3.2, 2.2], [0.5, 0.5, 1.6]), fillcolor = "#fef08a", linecolor =
"#ca8a04", linewidth = 2)
plot!(p8, [0.6, 1.6], [2.0, 2.5], color = "#16a34a", linewidth = 2.5)
scatter!(p8, [0.8], [1.0], color = "#dc2626", markersize = 8)

plot(
    p1, p2, p3, p4, p5, p6, p7, p8,
    layout = (4, 2),
    size = (700, 950)
)

catch fallo_proceso
println("Error durante la extracción o graficación en Julia: ", fallo_proceso)

plot(
    title = "Fallo en ejecución de bloque espacial",

```

```

framestyle = :none,
annotations = (0.5, 0.5, text("Error: " * string(fallo_proceso), 10, :red, :center
)),
legend = false,
size = (700, 950)
)
end

```

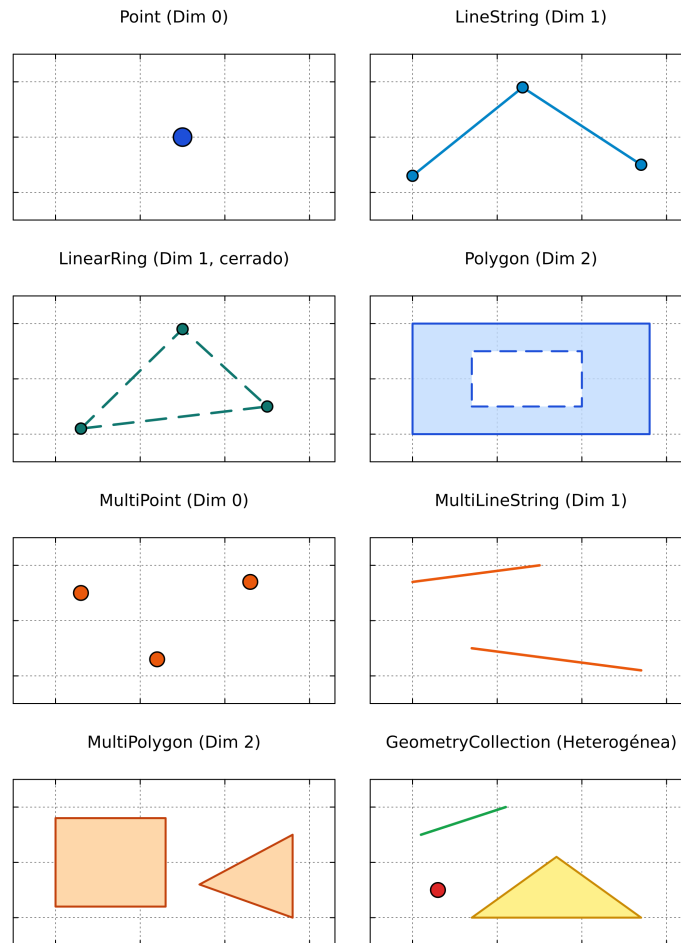


Figure 13: Instanciación espacial y morfología de tipos geométricos según el estándar OGC en Julia.

Catálogos de metadatos espaciales

En un motor relacional espacial, el clúster administra la información cartográfica y el esquema de tipos geométricos siguiendo las especificaciones normativas de la norma ISO 19125-2 / OGC Simple Feature Access - Part 2: SQL Option [ogc_sfa_part2_2010]. El estándar define dos estructuras de metadatos cardinales para la interoperabilidad entre motores de bases de datos y clientes SIG (Figura 3 de OGC 06-104r4):

- **spatial_ref_sys**: Diccionario canónico de sistemas de referencia de coordenadas (CRS). Almacena las proyecciones, elipsoides, datums y parámetros de transformación matemática.
- **geometry_columns**: Inventario descriptivo de columnas espaciales. Registra la relación entre tablas de entidades, nombres de columnas, dimensiones de coordenadas y el SRID asignado. Conforme a la cláusula 7.1.1 de la norma, los motores pueden implementar este catálogo como una vista dinámica sobre el diccionario del sistema en lugar de una tabla física base.

El catálogo spatial_ref_sys

El catálogo `spatial_ref_sys` materializa la especificación formal descrita en la cláusula 7.1.2 de la norma OGC SFA Parte 2, la cual exige la persistencia de los identificadores y definiciones WKT de los marcos geodésicos:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_cols="
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'spatial_ref_sys';
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_cols}"
```

column_name	data_type
srid	integer
auth_name	character varying
auth_srid	integer
srttext	character varying
proj4text	character varying

(5 rows)

Sus columnas clave comprenden:

- **srid**: Identificador entero del sistema de referencia espacial (habitualmente alineado con el estándar internacional EPSG).
- **auth_name**: Nombre del organismo emisor de la norma (comúnmente 'EPSG').
- **auth_srid**: Código numérico asignado por la autoridad cartográfica.
- **srtext**: Definición formal en formato WKT CRS (*Well-Known Text of Spatial Reference Systems*), requerida por clientes SIG y bibliotecas de procesamiento geométrico.
- **proj4text**: Cadena de inicialización de parámetros empleada por el motor de transformación cartográfica PROJ.

Al consultar las definiciones preinstaladas para Colombia, el motor lista los marcos geodésicos históricos y las fajas Gauss-Krüger tradicionales:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_sistemas_colombia="
SELECT
    srid,
    auth_name,
    split_part(srtext, '\\"', 2) AS nombre_sistema
FROM spatial_ref_sys
WHERE srtext ILIKE '%Colombia%' OR srtext ILIKE '%MAGNA-SIRGAS%'
ORDER BY srid;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_sistemas_colombia}"
```

srid	auth_name	nombre_sistema
3114	EPSG	MAGNA-SIRGAS / Colombia Far West zone
3115	EPSG	MAGNA-SIRGAS / Colombia West zone
3116	EPSG	MAGNA-SIRGAS / Colombia Bogota zone
3117	EPSG	MAGNA-SIRGAS / Colombia East Central zone
3118	EPSG	MAGNA-SIRGAS / Colombia East zone
4686	EPSG	MAGNA-SIRGAS
4996	EPSG	MAGNA-SIRGAS
4997	EPSG	MAGNA-SIRGAS

9377	EPSG	MAGNA-SIRGAS / Origen-Nacional
21891	EPSG	Bogota 1975 / Colombia West zone (deprecated)
21892	EPSG	Bogota 1975 / Colombia Bogota zone (deprecated)
21893	EPSG	Bogota 1975 / Colombia East Central zone (deprecated)
21894	EPSG	Bogota 1975 / Colombia East (deprecated)
21896	EPSG	Bogota 1975 / Colombia West zone
21897	EPSG	Bogota 1975 / Colombia Bogota zone
21898	EPSG	Bogota 1975 / Colombia East Central zone
21899	EPSG	Bogota 1975 / Colombia East
102231	ESRI	Colombia_West_West_Zone
102233	ESRI	MAGNA_Ciudad_Bogota

(19 rows)

⚠ Ausencia del código EPSG 9377 en volcados iniciales de spatial_ref_sys

La tabla `spatial_ref_sys` es una tabla relacional física en disco poblada a partir de los scripts iniciales de PostGIS o volcados históricos (como `init_unal_spatialdb.sql`). A diferencia del software de escritorio moderno o la biblioteca PROJ (que actualizan sus parámetros en cada compilación contra `proj.db`), el catálogo relacional en PostgreSQL es estático y no se actualiza de forma automática.

El sistema oficial de Colombia con Origen Nacional Único fue adoptado por el Instituto Geográfico Agustín Codazzi mediante la Resolución IGAC 471 de 2020 e indexado en el registro internacional bajo el código **EPSG:9377**. Si el clúster se inicializó con un esquema previo a dicha norma, la tupla no existirá en disco.

Aunque el tipado declarativo de PostgreSQL permite crear columnas con `geometry(Point, 9377)` sin validar la tabla, cualquier operación analítica de cálculo o reproyección cartográfica (`ST_Transform`) fallará en tiempo de ejecución si el registro no reside en `spatial_ref_sys`.

Como labor fundamental de administración de bases de datos (DBA), el registro oficial debe ser incorporado al catálogo. **Esta instrucción DML solo requiere ejecutarse una única vez en la base de datos**, pues la cláusula `ON CONFLICT` garantiza su persistencia permanente sin duplicación:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_insert_9377="
INSERT INTO spatial_ref_sys (srid, auth_name, auth_srid, srtext, proj4text)
VALUES (
    9377,
    'EPSG',
    9377,
```

```

        'PROJCS["MAGNA-SIRGAS / Origen-Nacional",GEOGCS["MAGNA-SIRGAS",DATUM["Marco_Geocent
        '+proj=tmerc +lat_0=4 +lon_0=-73 +k=0.9992 +x_0=5000000 +y_0=2000000 +ellps=GRS80 +towgs
    )
    ON CONFLICT (srid) DO UPDATE
    SET srtext = EXCLUDED.srtext, proj4text = EXCLUDED.proj4text;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
    -c "${query_insert_9377}"

```

```
INSERT 0 1
```

Mediante la cláusula `ON CONFLICT (srid) DO UPDATE`, la sentencia formaliza una operación atómica de tipo **UPSERT**, confiriendo **idempotencia** a la ejecución:

- **Primera compilación:** Al no existir el identificador 9377, el motor ejecuta la inserción estándar (`INSERT`) en el catálogo físico.
- **Compilaciones sucesivas:** El motor detecta la colisión sobre el índice de la clave primaria (`srid`) y, en lugar de abortar la transacción por infracción de unicidad (`duplicate key value violates unique constraint`), actualiza los parámetros sobre la tupla pre-existente.

Este diseño garantiza que los ciclos continuos de renderizado en Quarto mantengan el catálogo en un estado consistente y libre de registros redundantes, sin interrumpir el flujo de publicación del texto técnico.

Una vez registrada la tupla en disco, la inspección detallada de los sistemas cartográficos de referencia para el país expone sus parámetros elipsoidales y de proyección:

```

export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_colombia="
SELECT srid, auth_name, auth_srid, srtext
FROM spatial_ref_sys
WHERE srid IN (4326, 3116, 9377)
ORDER BY srid;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
    -x -P format=wrapped -P columns=90 \
    -c "${query_colombia}"

```

```

-[ RECORD 1 ]-----
srid      | 3116
auth_name | EPSG
auth_srid | 3116
srtext    | PROJCS["MAGNA-SIRGAS / Colombia Bogota zone",GEOGCS["MAGNA-SIRGAS",DATUM["Mar.
|.co_Geocentrico_Nacional_de_Referencia",SPHEROID["GRS 1980",6378137,298.257222.
|.101,AUTHORITY["EPSG","7019"]],TOWGS84[0,0,0,0,0,0,0],AUTHORITY["EPSG","6686"]
|.],PRIMEM["Greenwich",0,AUTHORITY["EPSG","8901"]],UNIT["degree",0.017453292519.
|.9433,AUTHORITY["EPSG","9122"]],AUTHORITY["EPSG","4686"]],PROJECTION["Transver.
|.se_Mercator"],PARAMETER["latitude_of_origin",4.596200416666666],PARAMETER["ce.
|.ntral_meridian",-74.07750791666666],PARAMETER["scale_factor",1],PARAMETER["fa.
|.lse_easting",1000000],PARAMETER["false_northing",1000000],UNIT["metre",1,AUTH.
|.ORITY["EPSG","9001"]],AUTHORITY["EPSG","3116"]]

-[ RECORD 2 ]-----
srid      | 4326
auth_name | EPSG
auth_srid | 4326
srtext    | GEOGCS["WGS 84",DATUM["WGS_1984",SPHEROID["WGS 84",6378137,298.257223563,AUTH.
|.ORITY["EPSG","7030"]],AUTHORITY["EPSG","6326"]],PRIMEM["Greenwich",0,AUTHORIT.
|.Y["EPSG","8901"]],UNIT["degree",0.0174532925199433,AUTHORITY["EPSG","9122"]],.
|.AUTHORITY["EPSG","4326"]]

-[ RECORD 3 ]-----
srid      | 9377
auth_name | EPSG
auth_srid | 9377
srtext    | PROJCS["MAGNA-SIRGAS / Origen-Nacional",GEOGCS["MAGNA-SIRGAS",DATUM["Marco_Ge.
|.ocentrico_Nacional_de_Referencia",SPHEROID["GRS 1980",6378137,298.257222101,A.
|.UTHORITY["EPSG","7019"]],AUTHORITY["EPSG","6686"]],PRIMEM["Greenwich",0,AUTHO.
|.RITY["EPSG","8901"]],UNIT["degree",0.0174532925199433,AUTHORITY["EPSG","9122".
|.]],AUTHORITY["EPSG","4686"]],PROJECTION["Transverse_Mercator"],PARAMETER["lat.
|.itude_of_origin",4],PARAMETER["central_meridian",-73],PARAMETER["scale_factor.
|. ",0.9992],PARAMETER["false_easting",5000000],PARAMETER["false_northing",20000.
|.00],UNIT["metre",1,AUTHORITY["EPSG","9001"]],AUTHORITY["EPSG","9377"]]

```

Entre estos identificadores fundamentales destacan:

- **SRID 4326:** Sistema geográfico global WGS 84 (coordenadas angulares en grados decimales de latitud y longitud).
- **SRID 3116:** Proyección MAGNA-SIRGAS / Colombia Bogotá (proyección conforme Gauss-Krüger sobre meridiano central 74°04'39.0285" W, con falso este de 1 000 000 m y falso norte de 1 000 000 m).
- **SRID 9377:** Proyección MAGNA-SIRGAS / Origen Nacional Único (sistema oficial conforme a la Resolución IGAC 471 de 2020, con origen en 4° N, 73° W, factor de escala

$k_0 = 0.9992$, falso este de 5 000 000 m y falso norte de 2 000 000 m).

La vista de catálogo `geometry_columns`

La especificación OGC SFA Parte 2 (cláusula 7.1.3) establece que toda relación que implemente atributos geométricos debe registrar sus metadatos descriptivos en el catálogo `GEOMETRY_COLUMNS` [`@ogc_sfa_part2_2010`]. Esta estructura provee a las aplicaciones clientes y motores de análisis la información indispensable para validar la dimensionalidad, el tipo de primitiva y el sistema de referencia de coordenadas sin necesidad de escanear la totalidad de las tuplas de la tabla.

En implementaciones tempranas de bases de datos espaciales, este catálogo operaba como una tabla física estática cuyo registro exigía procedimientos imperativos como `AddGeometryColumn()` o disparadores (*triggers*). A partir de PostGIS 2.0, y en concordancia con la cláusula 7.1.1 de la norma, `geometry_columns` se materializa como una vista del sistema construida dinámicamente por introspección sobre los catálogos relacionales de PostgreSQL (`pg_class`, `pg_attribute`, `pg_type`, `pg_namespace` y `pg_constraint`).

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_geom_cols="
SELECT f_table_schema, f_table_name, f_geometry_column, coord_dimension, srid, type
FROM geometry_columns
LIMIT 10;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_geom_cols}"
```

f_table_schema	f_table_name	f_geometry_column	coord_dimension	srid	type
base750	hidro_lineas	the_geom	2	21892	GEOMETRY
base750	vias_lineas	the_geom	2	21892	GEOMETRY
base750	vias_puntos	the_geom	2	21892	ST_Point
base750	vias_texto	the_geom	2	21892	ST_Point
divpol	departamentos	the_geom	2	21892	GEOMETRY
divpol	municipios	the_geom	2	21892	GEOMETRY
divpol	capa_departame. .ntos	the_geom	2	21892	GEOMETRY
divpol	capa_municipio. .s	the_geom	2	21892	GEOMETRY

public	infraestructur.	geom		2	9377	POINT
	.a_referencia					
public	muestras_geome.	geom		2	0	GEOMETRY
	.trias_ogc					

(10 rows)

Los campos descriptivos obligatorios según la norma OGC son:

- **f_table_schema**: Esquema relacional donde reside la tabla (ej. public, base750, divpol).
- **f_table_name**: Nombre de la tabla que contiene la capa espacial.
- **f_geometry_column**: Nombre de la columna que almacena los objetos espaciales (convencionalmente geom o the_geom).
- **coord_dimension**: Dimensión de las coordenadas almacenadas (2 para XY, 3 para XYZ/XYM, o 4 para XYZM).
- **srid**: Identificador entero del sistema de referencia espacial asociado.
- **type**: Restricción del tipo geométrico (POINT, LINESTRING, POLYGON, o GEOMETRY si admite tipos mixtos).

Para realizar una auditoría integral de la base de datos pedagógica **sig_db_unal**, se combina la vista **geometry_columns** con la tabla **spatial_ref_sys**. Esta consulta consolida los marcos de referencia en uso real, identificando las capas cartográficas existentes por esquema y detectando capas sin proyección asignada (**srid**: 0):

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_crs_en_uso="
SELECT
    g.srid,
    COALESCE(s.auth_name, 'LOCAL') AS autoridad,
    COALESCE(split_part(s.srtext, '\'', 2), 'Sin CRS asignado (Plano / Indefinido)') AS desc,
    count(*) AS numero_de_capas,
    string_agg(g.f_table_schema || '.' || g.f_table_name, ', ') AS capas_asociadas
FROM geometry_columns g
LEFT JOIN spatial_ref_sys s ON g.srid = s.srid
GROUP BY g.srid, s.auth_name, s.srtext
ORDER BY g.srid;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-x -P format=wrapped -P columns=90 \
-c "${query_crs_en_uso}"
```

```

-[ RECORD 1 ]-----+-----
srid          | 0
autoridad     | LOCAL
descripcion_crs | Sin CRS asignado (Plano / Indefinido)
numero_de_capas | 1
capas_asociadas | public.muestras_geometrias_ogc
-[ RECORD 2 ]-----+-----
srid          | 9377
autoridad     | EPSG
descripcion_crs | MAGNA-SIRGAS / Origen-Nacional
numero_de_capas | 1
capas_asociadas | public.infraestructura_referencia
-[ RECORD 3 ]-----+-----
srid          | 21892
autoridad     | EPSG
descripcion_crs | Bogota 1975 / Colombia Bogota zone (deprecated)
numero_de_capas | 8
capas_asociadas | base750.vias_puntos, base750.vias_texto, divpol.departamentos, base750..
                  | .hidro_lineas, divpol.capa_departamentos, divpol.capa_municipios, divpol.
                  | ..municipios, base750.vias_lineas

```

Asignación de identificador espacial en las vistas del esquema

Al examinar los resultados de la consulta cruzada anterior, se constata que las relaciones `capa_departamentos` y `capa_municipios` del esquema `divpol` reportan `srid: 0`. Esto ocurre porque ambos objetos son vistas relacionales (consultas almacenadas) en las que la columna geométrica fue proyectada sin un identificador de referencia explícito, lo que ocasiona que los clientes SIG interpreten la proyección como indefinida.

Aunque los fundamentos formales de DDL y DML para el diseño avanzado de vistas espaciales se estudiarán detalladamente en capítulos posteriores, es indispensable regularizar el catálogo desde esta primera sesión. La solución consiste en redefinir ambas vistas forzando la conversión explícita de la columna geométrica al sistema de coordenadas de la cartografía base del curso (EPSG:21892, MAGNA-SIRGAS / Colombia Bogotá):

```

export PGPASSWORD="${LOCAL_PGPASSWORD}"

# Redefinición de vistas con asignación explícita de SRID 21892
query_recreacion_vistas="
DROP VIEW IF EXISTS divpol.capa_departamentos;
CREATE VIEW divpol.capa_departamentos AS
SELECT

```

```

        d.gid,
        d.gid AS oid,
        d.dep_id AS id_dane,
        d.dep_nombre AS departamento,
        d.the_geom::geometry(Geometry, 21892) AS the_geom
FROM divpol.departamentos d
WHERE d.the_geom IS NOT NULL
ORDER BY d.dep_id;

```

```

DROP VIEW IF EXISTS divpol.capa_municipios;
CREATE VIEW divpol.capa_municipios AS
SELECT
    m.gid,
    m.gid AS oid,
    m.mun_id AS id_dane,
    d.dep_id AS dep_codigo_dane,
    initcap(d.dep_nombre::text) AS departamento,
    m.mun_nombre AS municipio,
    m.the_geom::geometry(Geometry, 21892) AS the_geom
FROM divpol.municipios m
LEFT JOIN divpol.departamentos d ON d.dep_id::text = m.dep_id::text
WHERE m.the_geom IS NOT NULL
ORDER BY m.mun_id;
"

```

```

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-c "${query_recreacion_vistas}"

```

```

DROP VIEW
CREATE VIEW
DROP VIEW
CREATE VIEW

```

Tras la ejecución del bloque anterior, se audita nuevamente el catálogo `geometry_columns` para validar la correcta inscripción del identificador espacial:

```

export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_verif_vistas="
SELECT
    f_table_schema,

```

```

    f_table_name,
    f_geometry_column,
    coord_dimension,
    srid,
    type
FROM geometry_columns
WHERE f_table_schema = 'divpol'
    AND f_table_name IN ('capa_departamentos', 'capa_municipios');
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_verif_vistas}"

```

f_table_schema	f_table_name	f_geometry_column	coord_dimension	srid	type
divpol	capa_departame. ntos	the_geom	2	21892	GEOMETRY
divpol	capa_municipio. s	the_geom	2	21892	GEOMETRY

(2 rows)

El catálogo reporta ahora el identificador 21892, garantizando la interoperabilidad con clientes de escritorio y servicios web.

Persistencia del catálogo mediante respaldo

Tras incorporar el sistema EPSG:9377 en el catálogo `spatial_ref_sys` y corregir el tipado de las vistas del esquema `divpol`, el catálogo relacional alcanza un estado consistente con el estándar OGC.

Para preservar estas modificaciones frente a eventuales destrucciones del volumen persistente, fallos del contenedor o reinicializaciones del clúster local, se recomienda generar un nuevo respaldo de la base de datos `sig_db_unal` aplicando los procedimientos documentados en la `?@sec-respaldo-recuperacion-bd`. Disponer de un archivo de volcado actualizado (en formato SQL plano o binario personalizado `.dump`) permite reconstruir la base de datos con las definiciones espaciales saneadas, eliminando la necesidad de reejecutar manualmente las sentencias de remediación en futuras restauraciones.

Verificación práctica en la consola integrada

A continuación se demuestra la secuencia operativa para comprobar la disponibilidad de la extensión espacial, aprovisionar una tabla pedagógica con tipado geométrico estricto, insertar una entidad mediante sintaxis WKT y auditar su catalogación automática:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

# 1. Inspección de extensiones espaciales activas
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
  -P format=wrapped -P columns=90 \
  -c "SELECT extname, extversion FROM pg_extension WHERE extname = 'postgis';"

# 2. Creación e inserción pedagógica (con limpieza previa para asegurar idempotencia)
query_ddl="
DROP TABLE IF EXISTS public.infraestructura_referencia;

CREATE TABLE public.infraestructura_referencia (
  id serial PRIMARY KEY,
  codigo varchar(10) NOT NULL UNIQUE,
  descripcion text,
  geom geometry(Point, 9377) NOT NULL
);

INSERT INTO public.infraestructura_referencia (codigo, descripcion, geom)
VALUES (
  'EST-001',
  'Estación de monitoreo central',
  ST_GeomFromText('POINT(5000000 2000000)', 9377)
);
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
  -c "${query_ddl}"

# 3. Auditoría de registro automático en geometry_columns
query_audit="
SELECT f_table_schema, f_table_name, f_geometry_column, coord_dimension, srid, type
FROM geometry_columns
WHERE f_table_name = 'infraestructura_referencia';
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
```

```
-P format=wrapped -P columns=90 \
-c "${query_audit}"
```

```
extname | extversion
-----+-----
postgis | 3.6.2
(1 row)
```

```
DROP TABLE
CREATE TABLE
INSERT 0 1
```

```
f_table_schema | f_table_name | f_geometry_column | coord_dimension | srid | type
-----+-----+-----+-----+-----+-----
public         | infraestructura_ref. | geom              | 2              | 9377 | POIN.
               | .erencia      |                   |                |     | .T
(1 row)
```

La consulta a `geometry_columns` reporta de forma automática la dimensión 2, el SRID 9377 y el tipo de dato `POINT`, validando la interoperabilidad del clúster con clientes SIG externos.

Lista de acrónimos

En la Table 5 se compilan las siglas y acrónimos técnicos empleados a lo largo del capítulo, indicando su equivalencia en inglés y su ámbito operativo.

Table 5: Catálogo de acrónimos y siglas técnicas normalizadas.

Acrónimo	Denominación formal en español	Término original en inglés	Ámbito funcional
ACID	Atomicidad, consistencia, aislamiento y durabilidad	<i>Atomicity, Consistency, Isolation, Durability</i>	Transaccionalidad en motores de bases de datos.
ADT	Tipo abstracto de datos	<i>Abstract Data Type</i>	Tipado extensible en modelos objeto-relacionales.
AIO	Entrada/salida asíncrona	<i>Asynchronous Input/Output</i>	Subsistema de almacenamiento del motor gestor.

Acrónimo	Denominación formal en español	Término original en inglés	Ámbito funcional
ANN	Vecinos más cercanos aproximados	<i>Approximate Nearest Neighbors</i>	Algoritmia de búsqueda en espacios vectoriales.
CBO	Optimizador basado en costos	<i>Cost-Based Optimizer</i>	Planificación y optimización de consultas SQL.
CRS	Sistema de referencia de coordenadas	<i>Coordinate Reference System</i>	Georreferenciación y geodesia computacional.
DBA	Administrador de base de datos	<i>Database Administrator</i>	Gestión y gobierno de la infraestructura del motor.
DDL	Lenguaje de definición de datos	<i>Data Definition Language</i>	Subconjunto SQL para definición de esquemas.
DML	Lenguaje de manipulación de datos	<i>Data Manipulation Language</i>	Subconjunto SQL para consulta y edición de tuplas.
ECEF	Centrado en la Tierra, fijo a la Tierra	<i>Earth-Centered, Earth-Fixed</i>	Sistema cartesiano tridimensional geocéntrico.
EPSG	Grupo europeo de prospección petrolífera	<i>European Petroleum Survey Group</i>	Registro canónico de parámetros geodésicos.
EWKB	Binario extendido bien conocido	<i>Extended Well-Known Binary</i>	Serialización binaria de geometrías con SRID.
GiST	Árbol de búsqueda generalizado	<i>Generalized Search Tree</i>	Estructura de indexación para dominios no escalares.
GQL	Lenguaje de consulta de grafos	<i>Graph Query Language</i>	Estándar formal ISO/IEC 39075 para bases de datos de grafos.
HNSW	Grafos jerárquicos navegables de mundo pequeño	<i>Hierarchical Navigable Small World</i>	Estructura de indexación y búsqueda aproximada de vectores.

Acrónimo	Denominación formal en español	Término original en inglés	Ámbito funcional
IGAC	Instituto Geográfico Agustín Codazzi	—	Autoridad cartográfica y geodésica de Colombia.
LSM	Árbol estructurado por fusión de registros	<i>Log-Structured Merge-tree</i>	Estructura de almacenamiento en motores de columnas anchas.
MBR	Rectángulo delimitador mínimo	<i>Minimum Bounding Rectangle</i>	Aproximación ortogonal planar de geometrías vectoriales.
MVCC	Control de concurrencia multiversión	<i>Multi-Version Concurrency Control</i>	Aislamiento transaccional sin bloqueos de lectura.
OGC	Consorcio geoespacial abierto	<i>Open Geospatial Consortium</i>	Organización de estandarización geoespacial internacional.
OLAP	Procesamiento analítico en línea	<i>Online Analytical Processing</i>	Cargas analíticas masivas y procesamiento columnar.
OLTP	Procesamiento de transacciones en línea	<i>Online Transaction Processing</i>	Transaccionalidad de alta concurrencia orientada a filas.
ORDBMS	Sistema gestor de bases de datos objeto-relacional	<i>Object-Relational Database Management System</i>	Modelo que combina álgebra relacional y tipos abstractos.
RBAC	Control de acceso basado en roles	<i>Role-Based Access Control</i>	Modelo de seguridad y asignación de privilegios en el motor.
SFS	Geometrías simples para SQL	<i>Simple Features for SQL</i>	Estándar OGC / ISO 19125 para tipos espaciales vectoriales.
SGBD	Sistema gestor de bases de datos	<i>Database Management System (DBMS)</i>	Infraestructura de software para administración de datos.

Acrónimo	Denominación formal en español	Término original en inglés	Ámbito funcional
SGBD espacial	Sistema gestor de bases de datos espaciales	<i>Spatial Database Management System (SDBMS)</i>	Motor relacional extendido para geometrías y topología.
SIG	Sistema de información geográfica	<i>Geographic Information System (GIS)</i>	Software para captura, edición y análisis cartográfico.
SIMD	Instrucción única, datos múltiples	<i>Single Instruction, Multiple Data</i>	Procesamiento vectorial a nivel de registros de la CPU.
SQL	Lenguaje de consulta estructurado	<i>Structured Query Language</i>	Estándar declarativo relacional internacional.
SQL/PGQ	Consultas de grafos de propiedades en SQL	<i>SQL/Property Graph Queries</i>	Parte 16 del estándar ISO SQL:2023.
SRID	Identificador del sistema de referencia espacial	<i>Spatial Reference Identifier</i>	Clave numérica para identificación unívoca de CRS.
SSI	Aislamiento de instantáneas serializables	<i>Serializable Snapshot Isolation</i>	Implementación de serializabilidad transaccional sin bloqueos.
TOAST	Técnica de almacenamiento de atributos sobredimensionados	<i>The Oversized-Attribute Storage Technique</i>	Mecanismo de compresión y desborde de páginas en PostgreSQL.
WAL	Bitácora de registro previo	<i>Write-Ahead Logging</i>	Mecanismo de persistencia y durabilidad transaccional.
WKB	Binario bien conocido	<i>Well-Known Binary</i>	Representación binaria estándar OGC para geometrías.
WKT	Texto bien conocido	<i>Well-Known Text</i>	Representación textual estándar OGC para geometrías y CRS.

Glosario de conceptos técnicos

- **Arquitectura canónica:** Modelo de referencia formal, teórico y universalmente aceptado en la literatura científica y los estándares internacionales para describir la estructura conceptual, la división de subsistemas y los principios operativos de una tecnología, con independencia de soluciones comerciales o implementaciones de código abierto particulares.
- **Atributo (o campo):** Unidad atómica nominada que define una característica, propiedad o dimensión de una entidad dentro de un dominio específico en el esquema relacional.
- **Atributo atómico frente a dato no atómico:** En el diseño relacional formal, clasifica los valores según su capacidad de descomposición estructural:
 - **Atributo atómico:** Valor escalar indivisible que carece de subestructuras con significado independiente dentro del modelo de datos (requisito indispensable de la primera forma normal o 1NF). Ejemplos: un código numérico `srid`, una marca temporal o un identificador entero.
 - **Dato no atómico:** Estructura compuesta, multivaluada o jerárquica cuyos componentes internos pueden descomponerse, direccionarse o evaluarse de forma individual sin alterar la validez del objeto. Ejemplos: arreglos (`ARRAY`), objetos semiestructurados (`JSONB`), tipos compuestos o entidades geométricas (`geometry`), conformadas internamente por secuencias ordenadas de pares o tripletas de coordenadas (X, Y, Z, M) y anillos topológicos.
- **Base de datos espacial (GeoDB):** Colección integrada y estructurada de datos geométricos, geodésicos y alfanuméricos persistidos bajo un esquema unificado, accesible de forma concurrente, con soporte de transaccionalidad ACID y gobernada por catálogos de referencia cartográfica.
- **Base de datos vectorial:** Sistema gestor de persistencia e indexación diseñado para almacenar coordenadas de alta dimensionalidad (*embeddings* en $\mathbb{R}^d$) y resolver consultas de afinidad mediante métricas formales de distancia (euclidiana, coseno o producto escalar) estructuradas sobre algoritmos de búsqueda aproximada de vecinos más cercanos (ANN).
- **Búsqueda por similitud semántica:** Método de recuperación de información que evalúa la afinidad conceptual entre entidades no estructuradas proyectadas hacia un espacio métrico latente continuo mediante modelos de aprendizaje profundo, sustituyendo la coincidencia léxica exacta por cálculo de proximidad geométrica multidimensional.
- **Cartucho de datos (*cartridge*) o extensión:** Paquete de software desacoplado que se acopla a las interfaces de extensibilidad de un motor objeto-relacional para dotarlo de soporte nativo hacia un dominio específico (por ejemplo, tipos espaciales, clases de

operadores GiST y funciones OGC provistas por la extensión `postgis`), sin requerir modificaciones ni recompilación en el código fuente nuclear del gestor.

- **Codificador (*encoder*) y representación densa (*embedding*):**
 - **Codificador (*encoder*):** Función de mapeo $f : \mathcal{X} \rightarrow \mathbb{R}^d$ implementada mediante una red neuronal profunda que transforma una entidad de entrada no estructurada desde un espacio disperso hacia un vector numérico denso en un espacio continuo de dimensión fija d .
 - **Representación densa (*embedding*):** Vector numérico resultante de la inferencia del codificador, donde la proximidad métrica relativa entre vectores condensa equivalencia o similitud semántica.
- **Ejecución vectorial (*vectorized execution*):** Paradigma de diseño del procesador de consultas en motores analíticos (OLAP) que sustituye el modelo iterador tupla a tupla por la evaluación de bloques homogéneos contiguos de valores de una misma columna en la memoria intermedia, permitiendo al compilador emitir instrucciones de hardware **SIMD** a nivel de registros de CPU.
- **Esquema de base de datos:** Definición estructural, lógica y semántica de las relaciones, tipos de datos, restricciones de dominio, claves y reglas de integridad que gobiernan un repositorio relacional, expresada formalmente mediante sentencias DDL.
- **Esquema en lectura (*schema-on-read*):** Paradigma de persistencia (característico de motores NoSQL documentales) donde los datos se almacenan sin someterse a una validación estructural previa contra un esquema DDL formal. El motor o la aplicación compila e interpreta la estructura, los tipos de datos y la jerarquía en el momento exacto de evaluar la consulta, contrastando con el esquema rígido en escritura (*schema-on-write*) propio de los sistemas relacionales.
- **Garantías ACID:** Conjunto de propiedades operativas que gobiernan el procesamiento transaccional en un SGBD:
 - **Atomicidad (*Atomicity*):** Garantía de indivisibilidad operativa; una transacción se consolida en su totalidad o revierte todas sus modificaciones mediante el uso coordinado del registro previo en bitácora (WAL) y registros de reversión (*rollback*).
 - **Consistencia (*Consistency*):** Preservación del estado válido del catálogo relacional; el motor invalida y aborta cualquier transacción que transgreda restricciones declarativas (PRIMARY KEY, FOREIGN KEY, CHECK) o disparadores.
 - **Aislamiento (*Isolation*):** Ejecución concurrente protegida contra anomalías; el gestor de transacciones aísla las modificaciones simultáneas valiéndose de control de concurrencia multiversión (MVCC) o protocolos de bloqueo.
 - **Durabilidad (*Durability*):** Garantía de persistencia no volátil; el motor asegura que, una vez confirmado el COMMIT, los cambios residan en disco persistente mediante sincronización del WAL (`fsync`), permitiendo su recuperación ante fallos del kernel o cortes eléctricos.

- **Grafos de mundo pequeño navegables y jerárquicos (HNSW):** Estructura de indexación basada en grafos multicapa superpuestos que optimiza la búsqueda aproximada de vecinos más cercanos (ANN) en espacios vectoriales continuos, logrando una complejidad temporal de escala logarítmica $O(\log N)$ mediante un recorrido voraz (*greedy routing*) descendente.
- **Modelo de despliegue embebido (*in-process*):** Arquitectura donde el motor de base de datos se vincula como biblioteca de ejecución dentro del mismo proceso y espacio de memoria de la aplicación anfitriona (DuckDB o SQLite), eliminando la comunicación por red y la sobrecarga de sockets propia del esquema cliente-servidor.
- **Relación (o tabla):** Estructura lógica bidimensional fundamental del modelo relacional, formalizada matemáticamente como un subconjunto no ordenado del producto cartesiano de los dominios de sus atributos, consistente con un esquema formal y compuesta por tuplas homogéneas.
- **Serialización:** Concepto con doble acepción en sistemas gestores de bases de datos:
 - **Serializabilidad transaccional:** Nivel supremo de aislamiento (**SERIALIZABLE**) que asegura que el resultado final de transacciones concurrentes sea equivalente a una ejecución estrictamente secuencial, previniendo anomalías complejas como el sesgo de escritura (*write skew*).
 - **Serialización de datos:** Proceso algorítmico de codificación que transforma estructuras dinámicas en memoria RAM (objetos C, topologías o grafos) en secuencias contiguas de bytes (como WKB/EWKB) o representaciones textuales (como WKT o GeoJSON) para su persistencia física o transporte en red.
- **Solución *ad-hoc*:** Implementación técnica construida a la medida para resolver una demanda analítica o de almacenamiento puntual, caracterizada por la ausencia de un marco arquitectónico reutilizable, carencia de esquemas formales y acoplamiento dependiente del formato local.
- **Tipo abstracto de datos (ADT):** Tipo de dato formal definido por el usuario o una extensión cuya estructura interna y métodos de serialización están encapsulados, operando en el motor como un tipo de dato nativo.
- **Tipo de dato geography:** Tipo abstracto de datos especializado en PostGIS para la persistencia y cómputo de relaciones topológicas sobre un elipsoide geodésico de referencia (comúnmente WGS 84, EPSG:4326). Opera bajo trigonometría esférica/elipsoidal, emitiendo resultados métricos en metros sin requerir transformaciones cartográficas planas intermedias, con una sobrecarga de cálculo superior al tipo plano **geometry**.
- **Tupla (registro o fila):** Unidad estructural elemental en una relación relacional que representa una instancia o entidad discreta del dominio, conformada por una colección finita de valores correspondientes a los atributos declarados en el esquema.